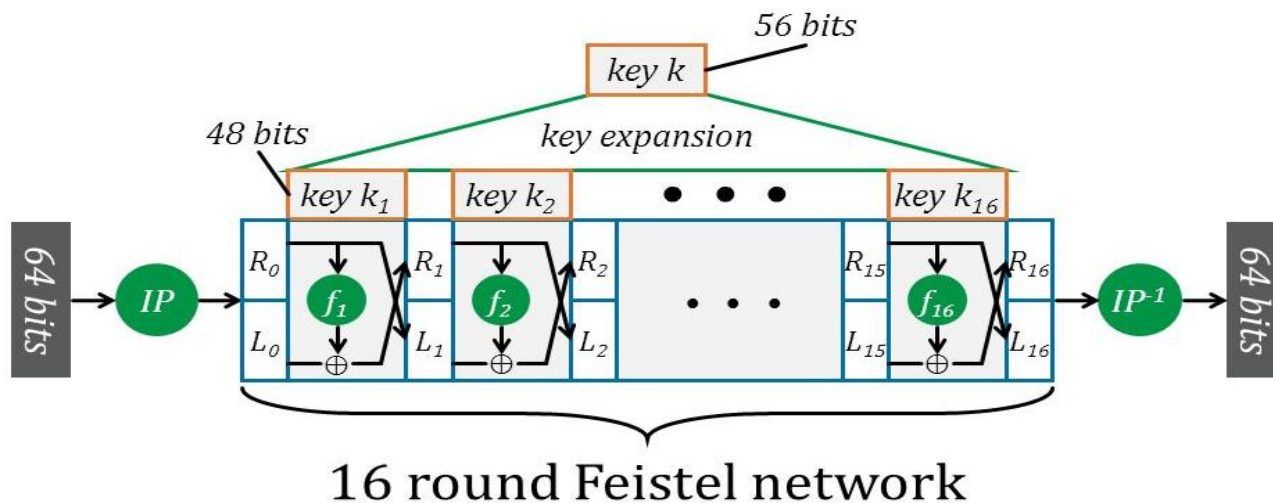
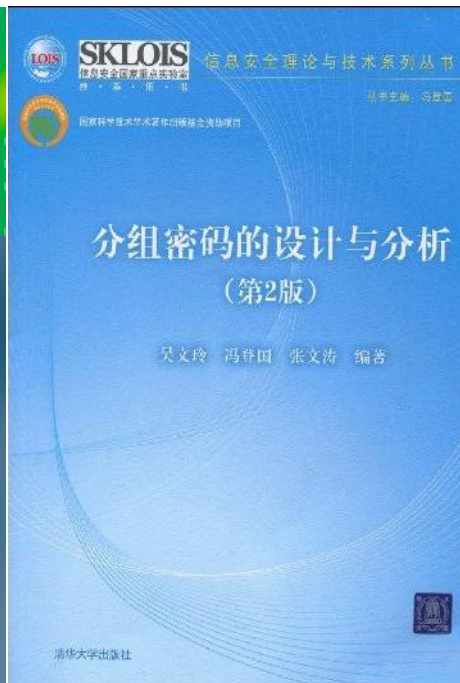
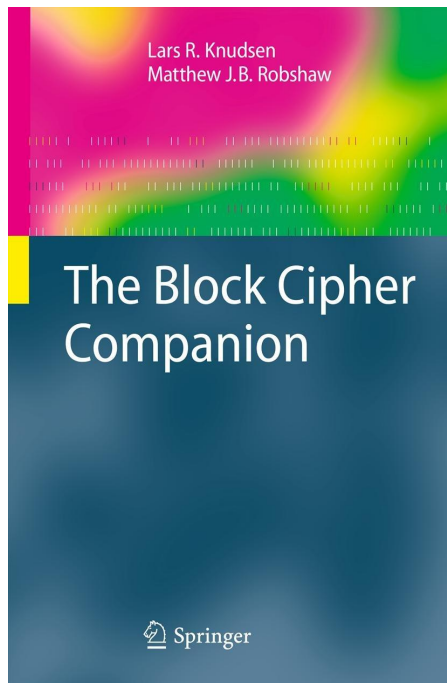




分组密码

杜育松

计算机学院

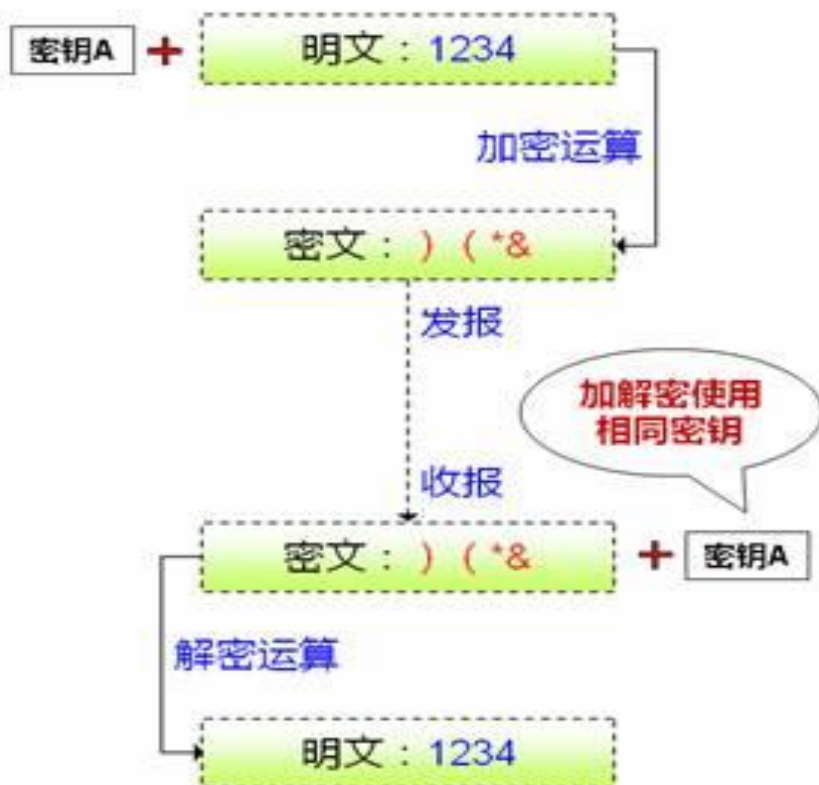
2025学年下学期

对称密码

- 现代密码学中加密可以分成两大类：
- 如果加密密钥和解密密钥相同（或者通过加密密钥容易推出解密密钥），则密码为对称密码。
- 如果加密密钥和解密密钥不相同，并且通过加密密钥推出解密密钥非常困难，则密码为非对称（公钥）密码。
- 对称密码又分为两类：
 - ① 分组密码(Block Ciphers)
 - ② 流密码(Stream Ciphers)，也被称为序列密码

对称密码

对称加密

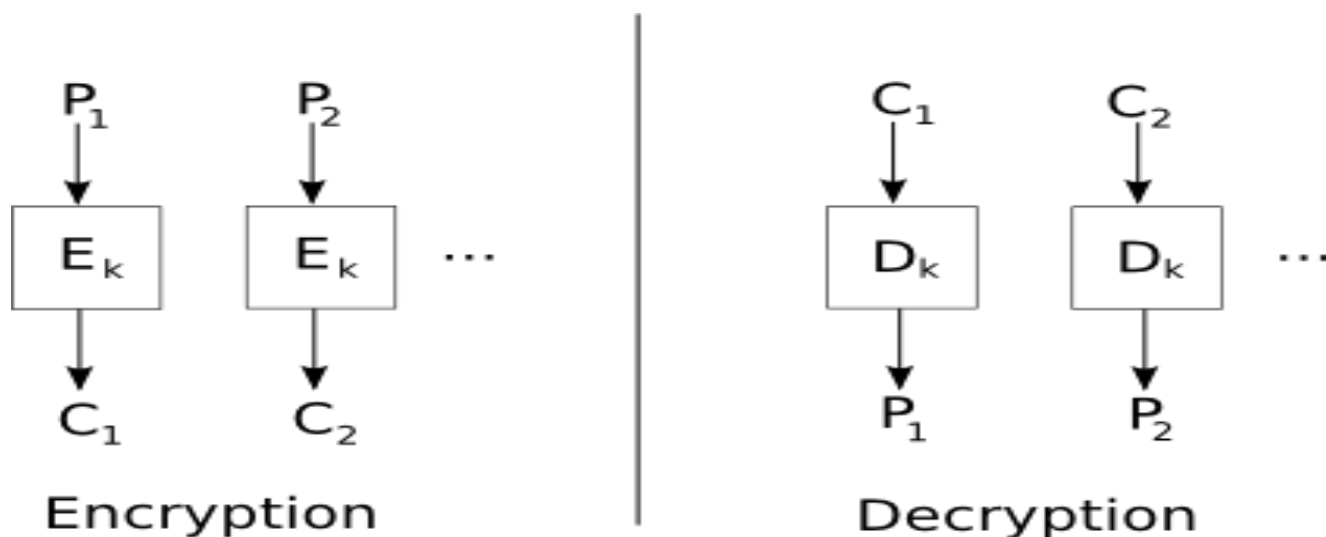


非对称加密



分组密码

- 分组密码是指将处理的明文按照固定的长度进行分组，加密解密的处理在一个固定长度密钥的控制下，以一个分组为单位独立进行，得出一个固定长度的对应于明文分组的密文分组。



使用 *OpenSSL* 中的分组加密算法

- *EVP_EncryptInit_ex* 用于初始化加密上下文，指定加密算法及模式（如 *AES* 算法 *CBC* 模式）并传入密钥、*IV*，为分组加密搭建核心环境；
- *EVP_EncryptUpdate* 可以多次被调用处理文件流式数据，按加密算法固定的分组长度（例如 *16* 字节）加密文件能包含的所有完整块；
- *EVP_EncryptFinal_ex* 通常仅在最后被调用一次，对文件中的剩余数据做 *PKCS7* 填充，以凑够固定的分组长度（例如 *16* 字节），完成最后一个分组的加密。

分组密码：DES

- 1976年11月，IBM研制并改版的Lucifer算法被采纳为美国联邦标准，批准用于非军事场合的各种政府机构。1977年2月15日，数据加密标准(DES)，即FIPS-PUB-46正式发布。
- DES是分组密码的典型代表，也是第一个被公布出来的加密标准算法。
- DES所使用的Feistel密码结构目前仍然被广泛研究和采用。

分组密码：从DES到AES

- 美国标准技术研究所NIST在1997年公开征集新的高级加密标准AES(Advanced Encryption Standards)。
- 1998年6月NIST共收到21个提交的算法，通过充分研究和筛选，2000年10月比利时研究人员研制的Rijndael算法被选为高级加密标准。2001年11月，NIST发布关于AES的FIPS-PUB-197标准。
- AES使用SPN（代换置换网络）结构，不同于DES所使用的Feistel结构，同样被广泛研究和采用。

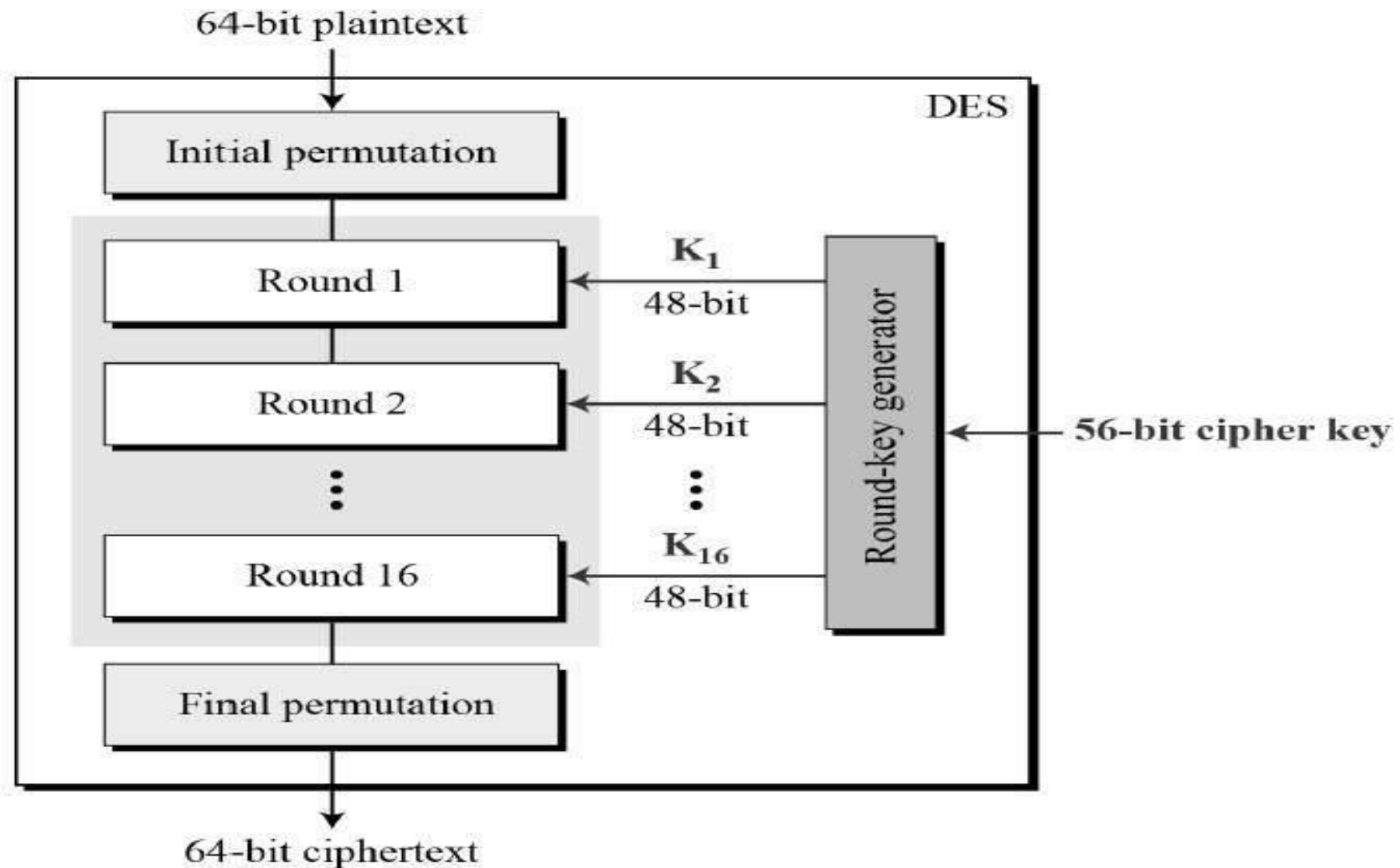
其他分组密码算法

- IDEA（International Data Encryption Algorithm）由我国的来学嘉教授（瑞士国籍）和瑞士的J.L.Massey教授于1990年提出。
- SMS4是由国家密码管理局组织研制于2006年公布的适用于无限局域网产品的建议密码算法。
- 更多的分组密码算法和相关知识？建议阅读
 1. 分组密码的设计与分析 吴文玲等 编著 清华大学出版社
 2. 图解密码技术 结成浩（日）著 人民邮电出版社
 3. Java加密与解密的艺术 梁栋 著 机械工业出版社
 4. Python密码学编程 Al Sweigart（美）著 人民邮电出版社

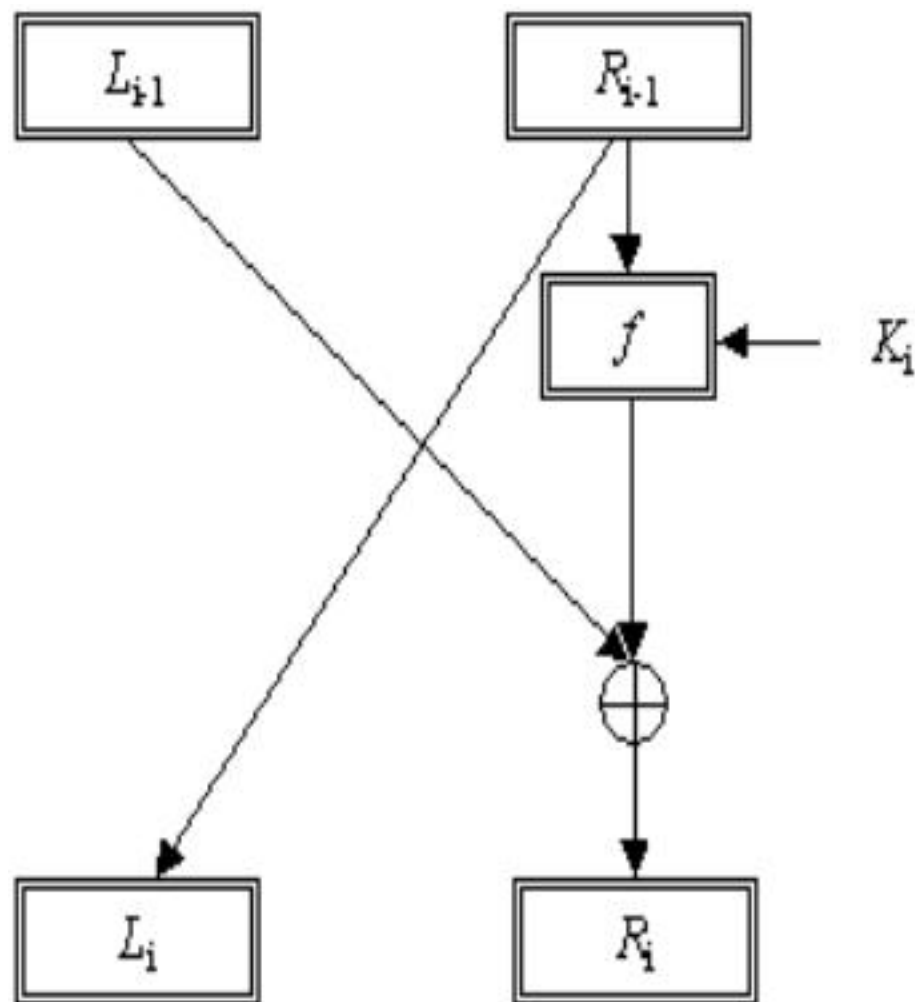
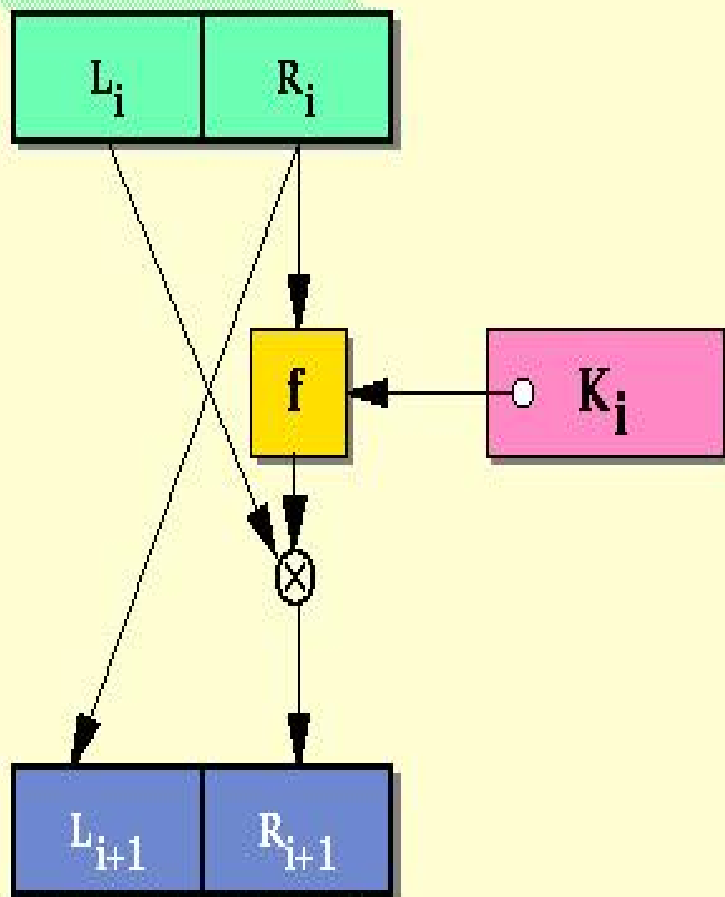
分组密码DES包含两个部件

- DES包含轮函数和密钥扩展算法两个部件。
- 轮函数：有两个部份组成
 - ① S-盒（S-Box，代换盒）：实现混淆
 - ② 扩散层：实现扩散
- 密钥扩展算法：将初始密钥扩展成为多个长度相同的子密钥。
- DES轮函数的结构被称为Feistel结构。DES是一个16轮的Feistel型密码。从算法实现的角度来看，它是以轮函数为循环体的16轮循环操作，每一轮循环需要使用一个由密钥扩展算法生成的子密钥。

DES是一个16轮的Feistel型密码



DES使用的Feistel密码结构



Feistel密码结构

- 在Feistel密码结构中，每一个状态 u^i 被分成相同长度的两半 L^{i-1} 和 R^{i-1} 。
- 轮函数 g 具有以下形式

$$g(L^{i-1}, R^{i-1}, K^i) = (L^i, R^i)$$

其中

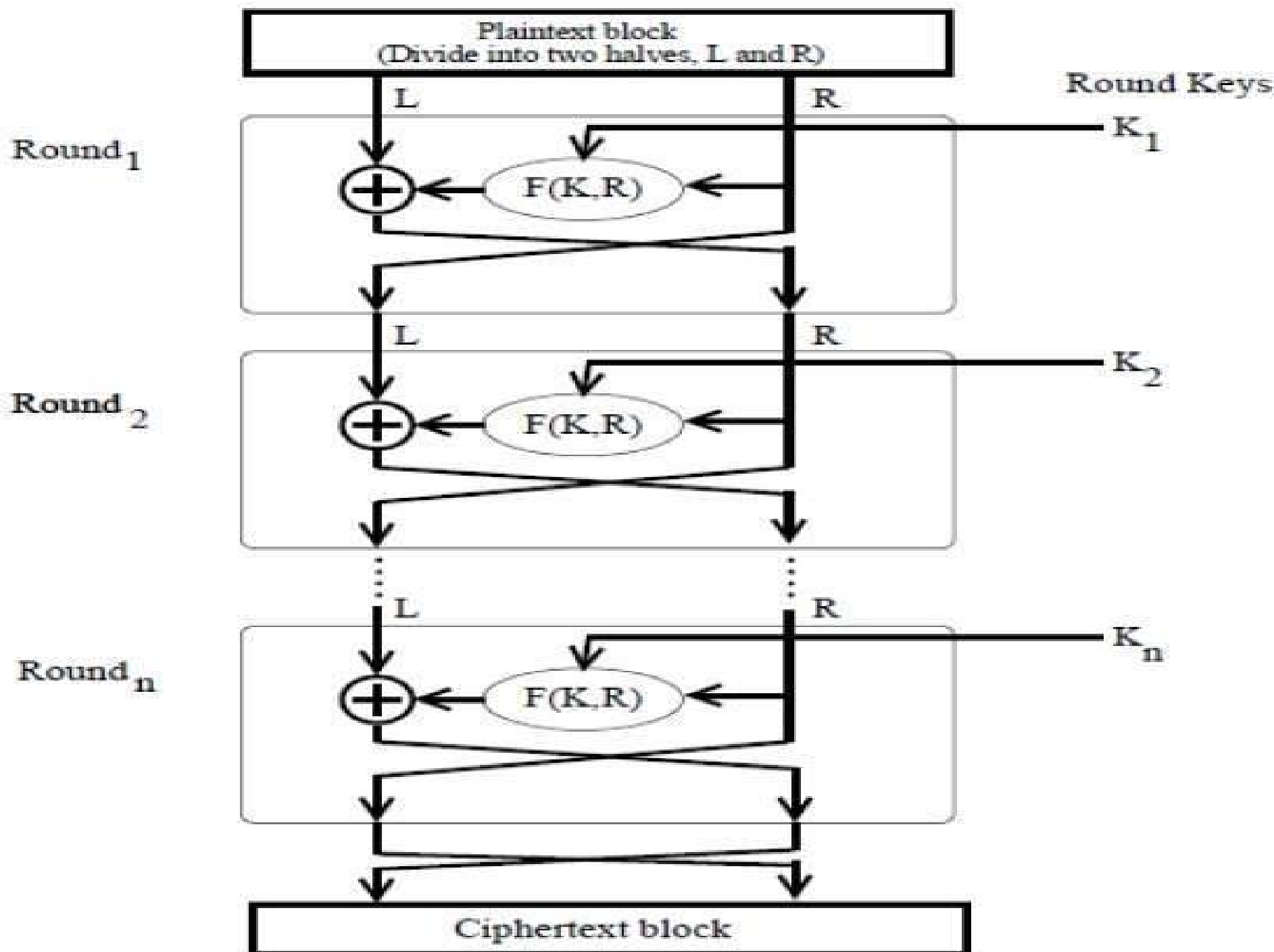
$$L^i = R^{i-1}$$

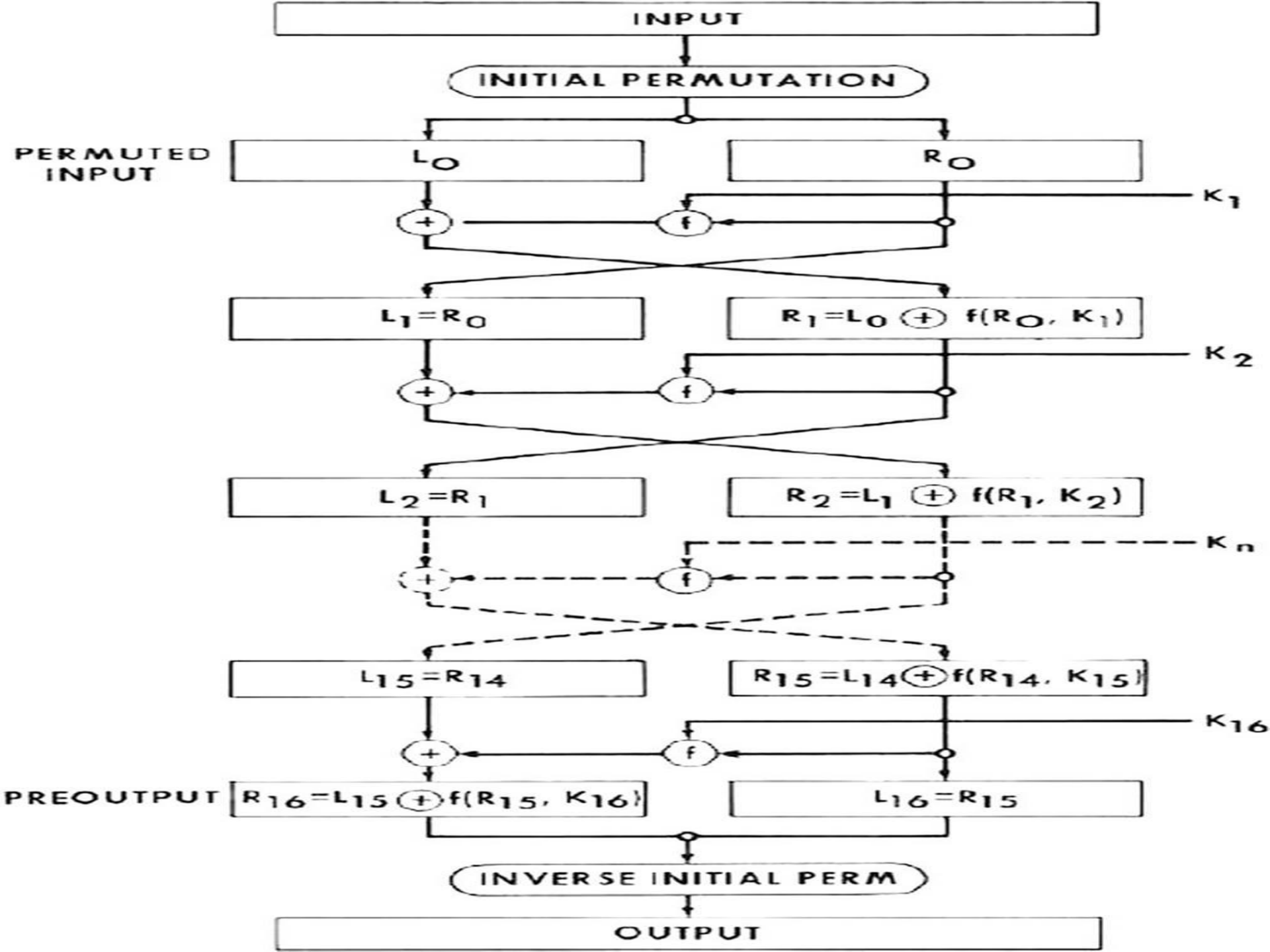
$$R^i = L^{i-1} \oplus f(R^{i-1}, K^i)$$

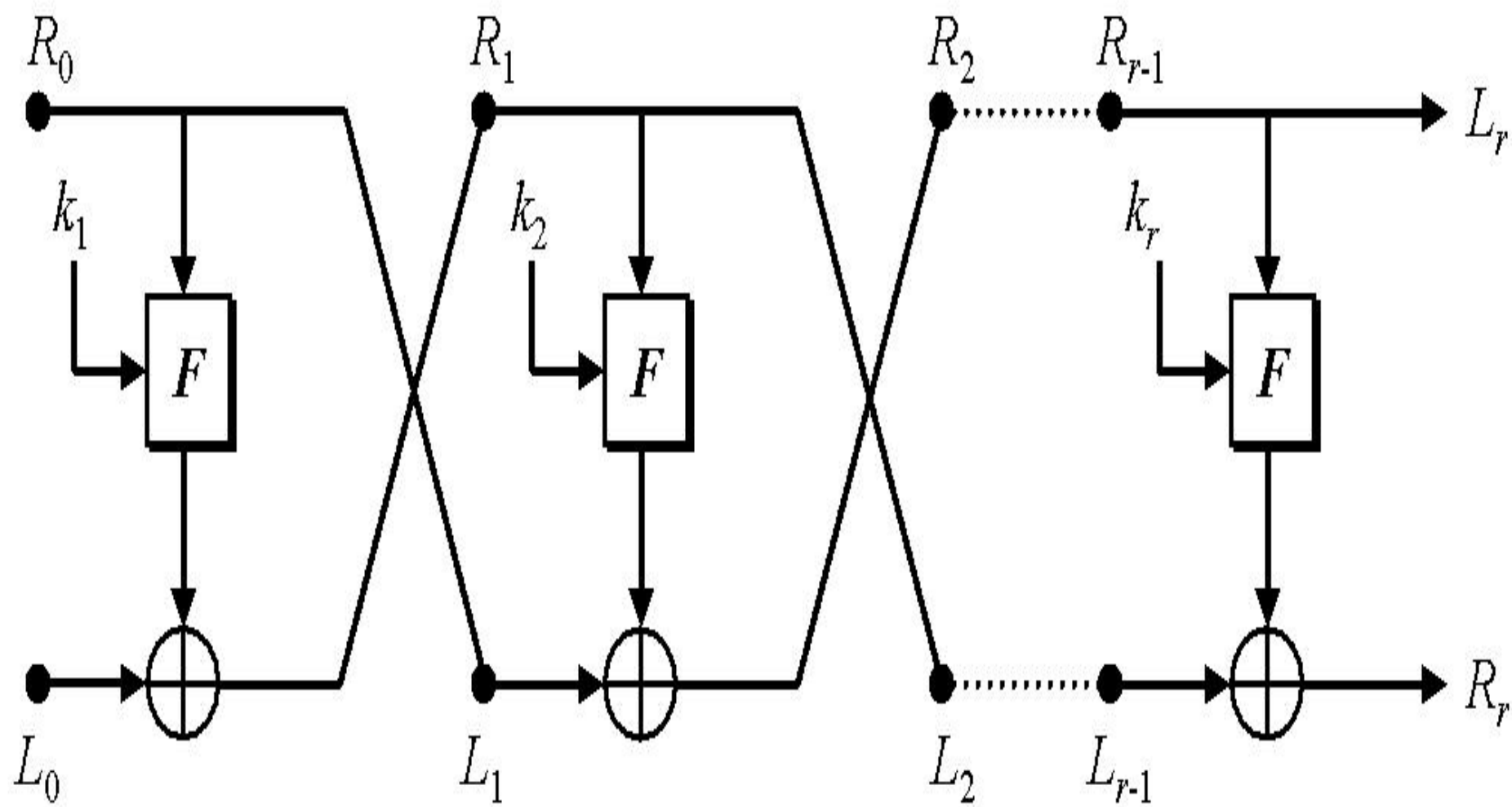
- K^i 被称为轮密钥，或子密钥，是由密钥扩展算法（密钥编排方案）在初始密钥 K 的基础上运算得到的。

分组密码DES的加密过程

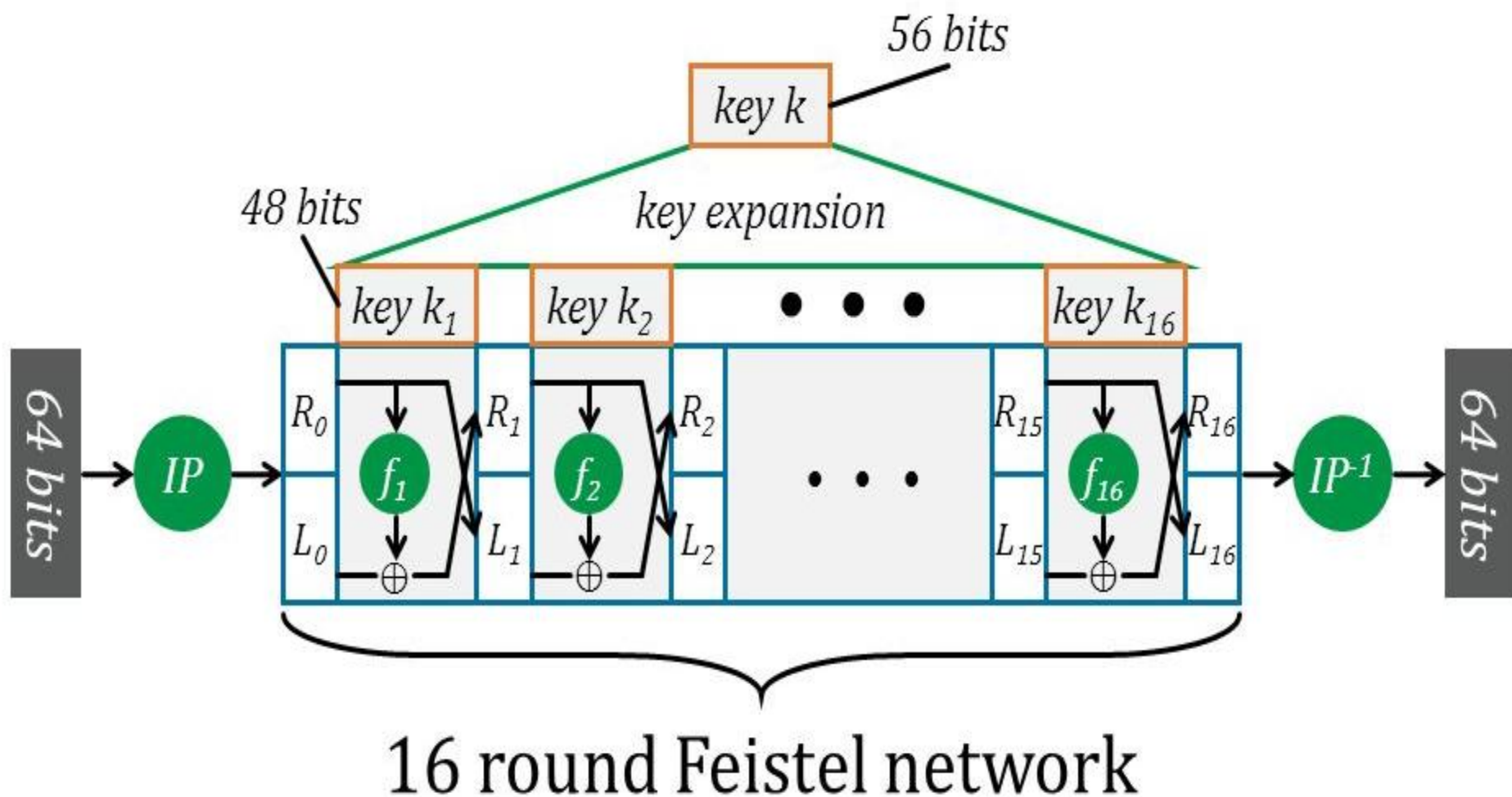
- DES的密钥长度为56比特，分组长度为64比特，即每次用56比特加密或解密64个比特。
- DES在16轮加密前先对明文 x 做一个固定长度的初始置换IP，得到 $IP(x)$ ，然后把 $IP(x)$ 分成左右两部分，即 $IP(x)=L^0R^0$ 。
- 以 L^0R^0 作为DES轮函数 g 的输入，分别使用长度均为48比特的 $K^1, K^2, \dots, K^{16}$ 作为每一轮的密钥，连续进行16轮运算，得到 $L^{16}R^{16}$ 。
- 将 $L^{16}R^{16}$ 左半部分和右半部分位置对调，做逆IP置换，得到密文 $y=IP^{-1}(R^{16}L^{16})$ 。







$$f_1, \dots, f_{16} : \{0, 1\}^{32} \rightarrow \{0, 1\}^{32} \text{ and } f_i(x) = \mathbf{F}(k_i, x)$$



分组密码*DES*的解密过程

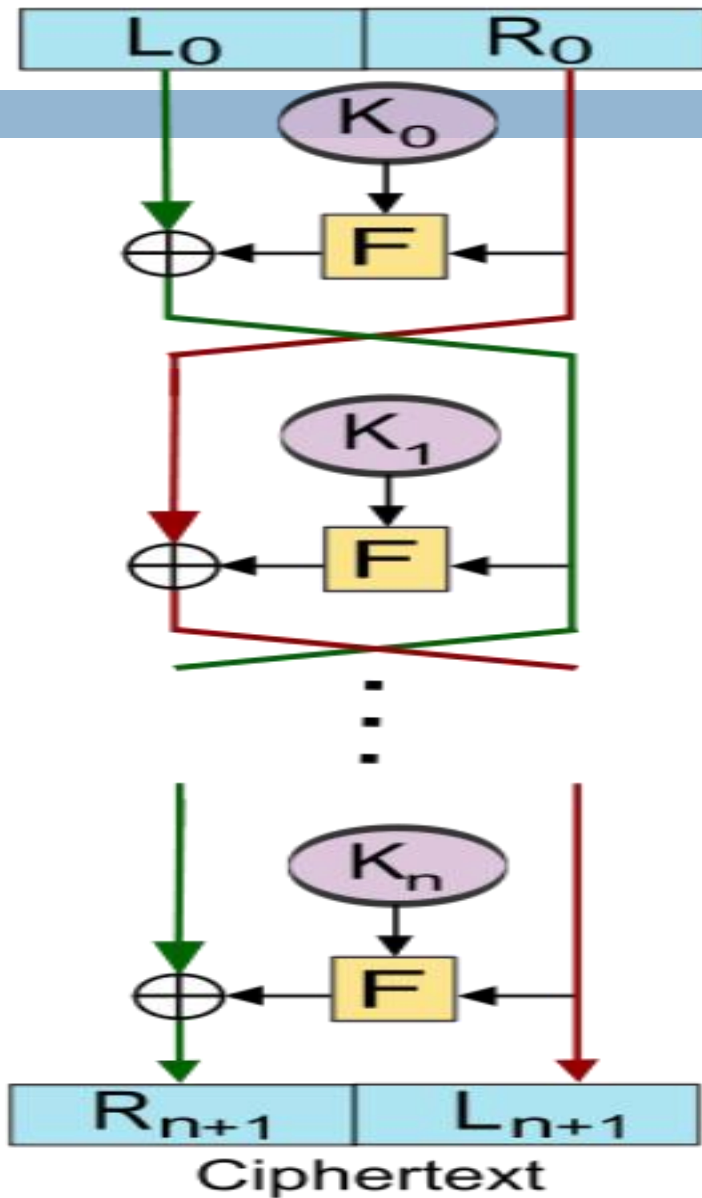
- DES的解密过程与加密过程几乎一样，只要把密文 $y=IP^{-1}(R^{16}L^{16})$ 作为输入，以逆序的方式使用密钥，即分别使用 $K^{16}, K^{15}, \dots, K^1$ 作为每一轮的密钥，输出结果就是明文 x 。
- DES的这种加密和解密过程几乎一样的特点是由DES的Feistel密码结构决定的，即

$$L^{i-1} = R^i \oplus f(R^i, K^i)$$

$$R^{i-1} = L^i$$

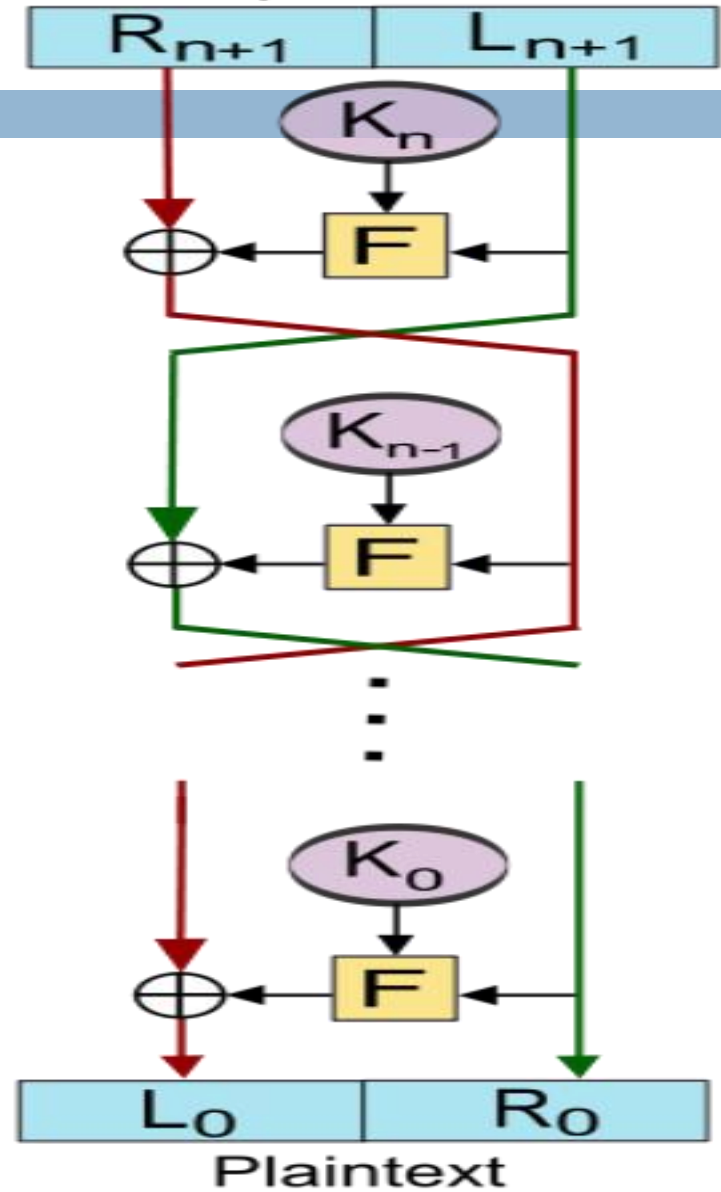
Encryption

Plaintext



Decryption

Ciphertext



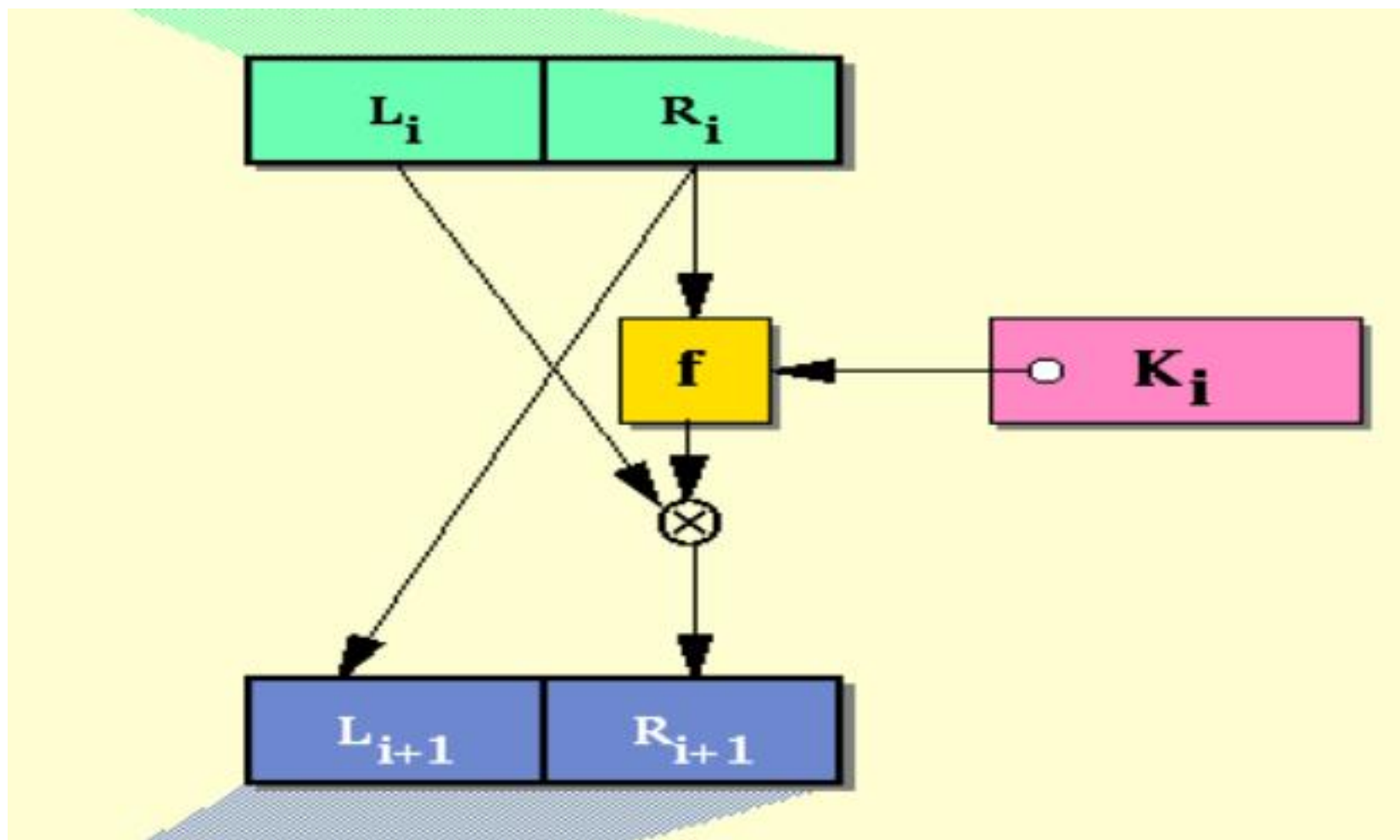
DES轮函数g中的f函数

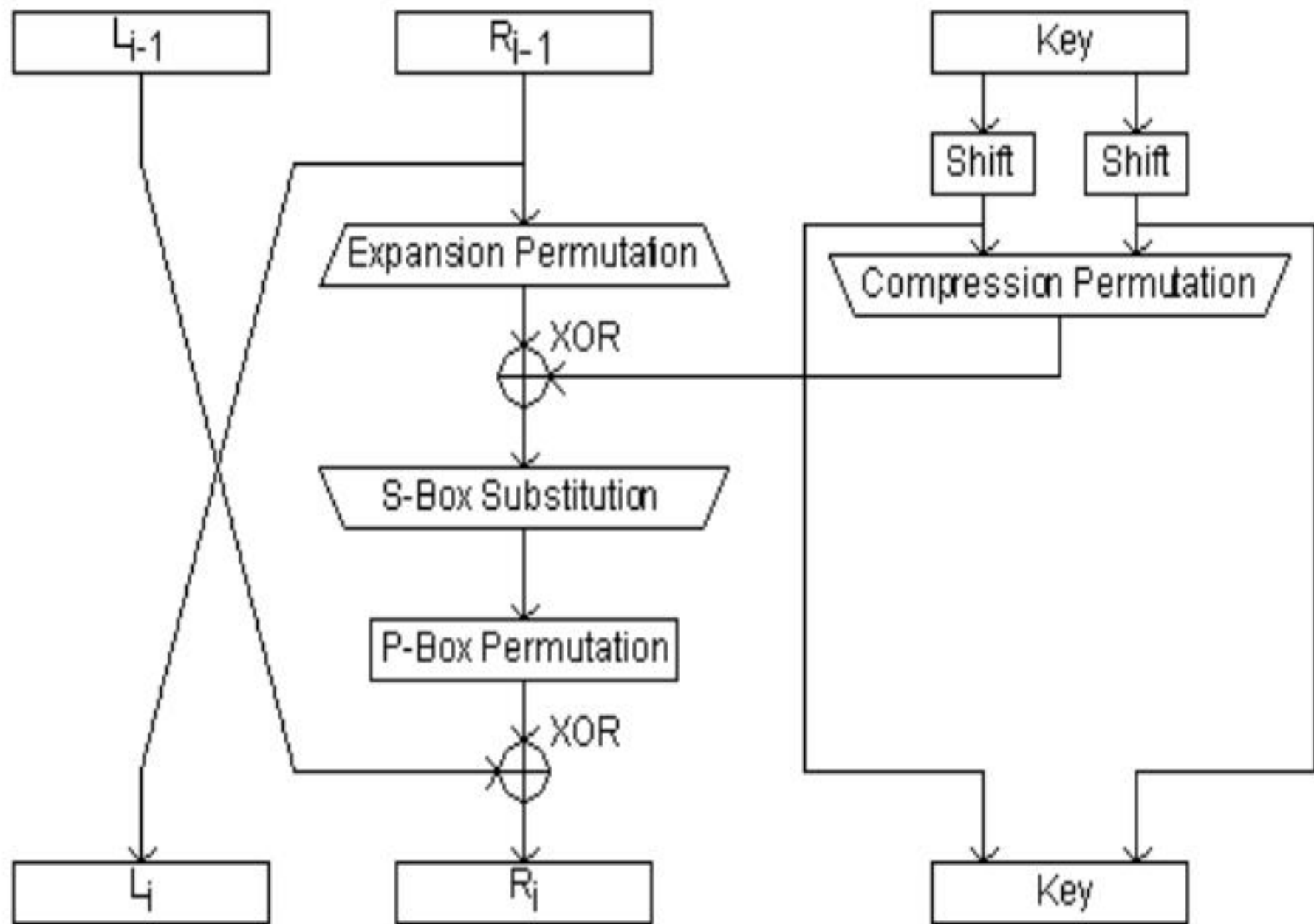
- DES的轮函数g包含一个通常被记为f的函数。
- 函数f以长度为32比特的串和长度为48比特的轮密钥作为输入，输出长度为32比特的串。这样的函数通常可以表示成：

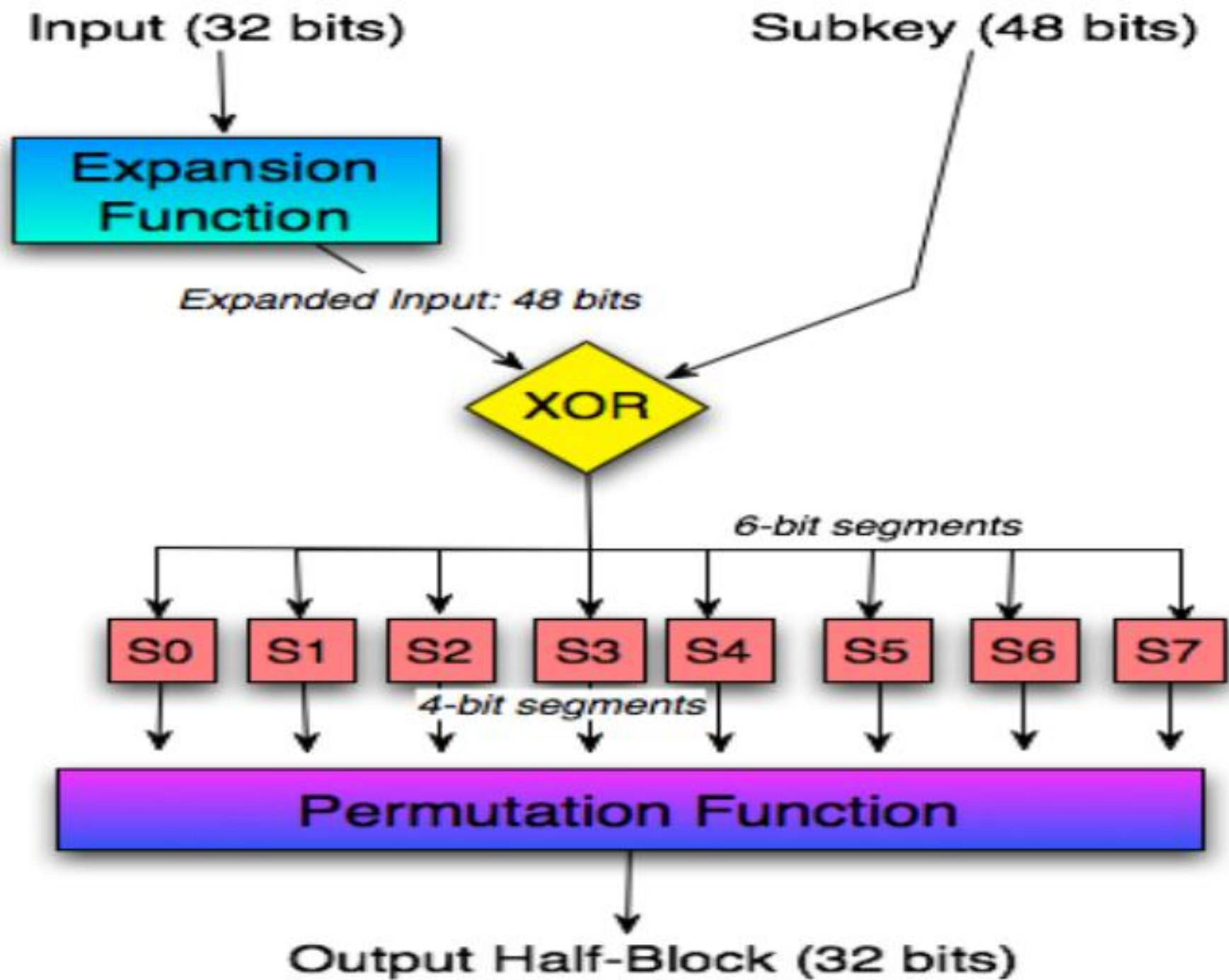
$$f:\{0,1\}^{32} \times \{0,1\}^{48} \rightarrow \{0,1\}^{32}$$

- 函数f包含一个扩张置换(Expansion Permutation)，一个S盒代换(S-Box Substitution)和一个P盒置换(P-Box Permutation)。
- 函数f是DES的核心，而S盒则是函数f的核心。

DES使用的Feistel密码结构







Half Block (32 bits)

Subkey (48 bits)

E



S1

S2

S3

S4

S5

S6

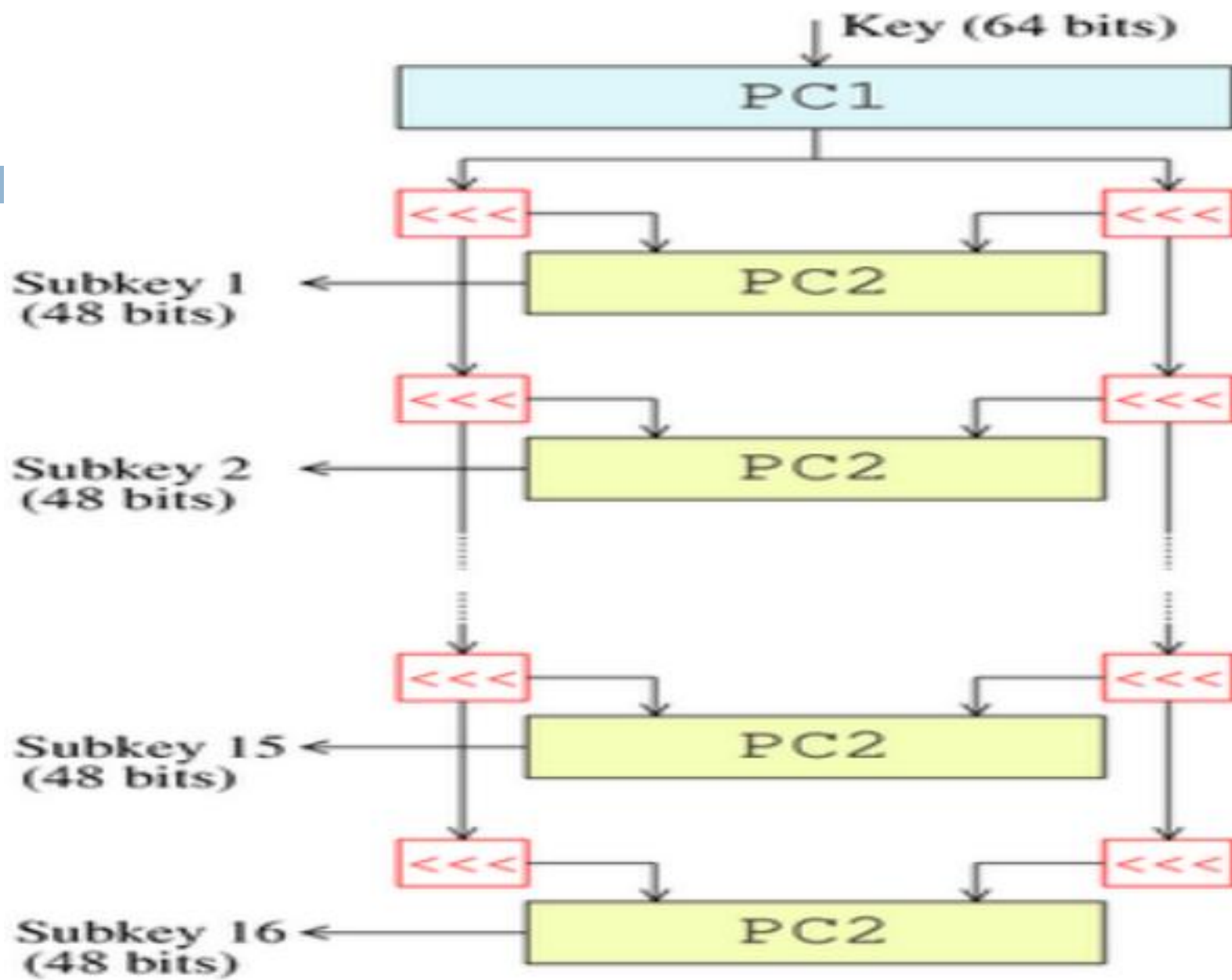
S7

S8

P

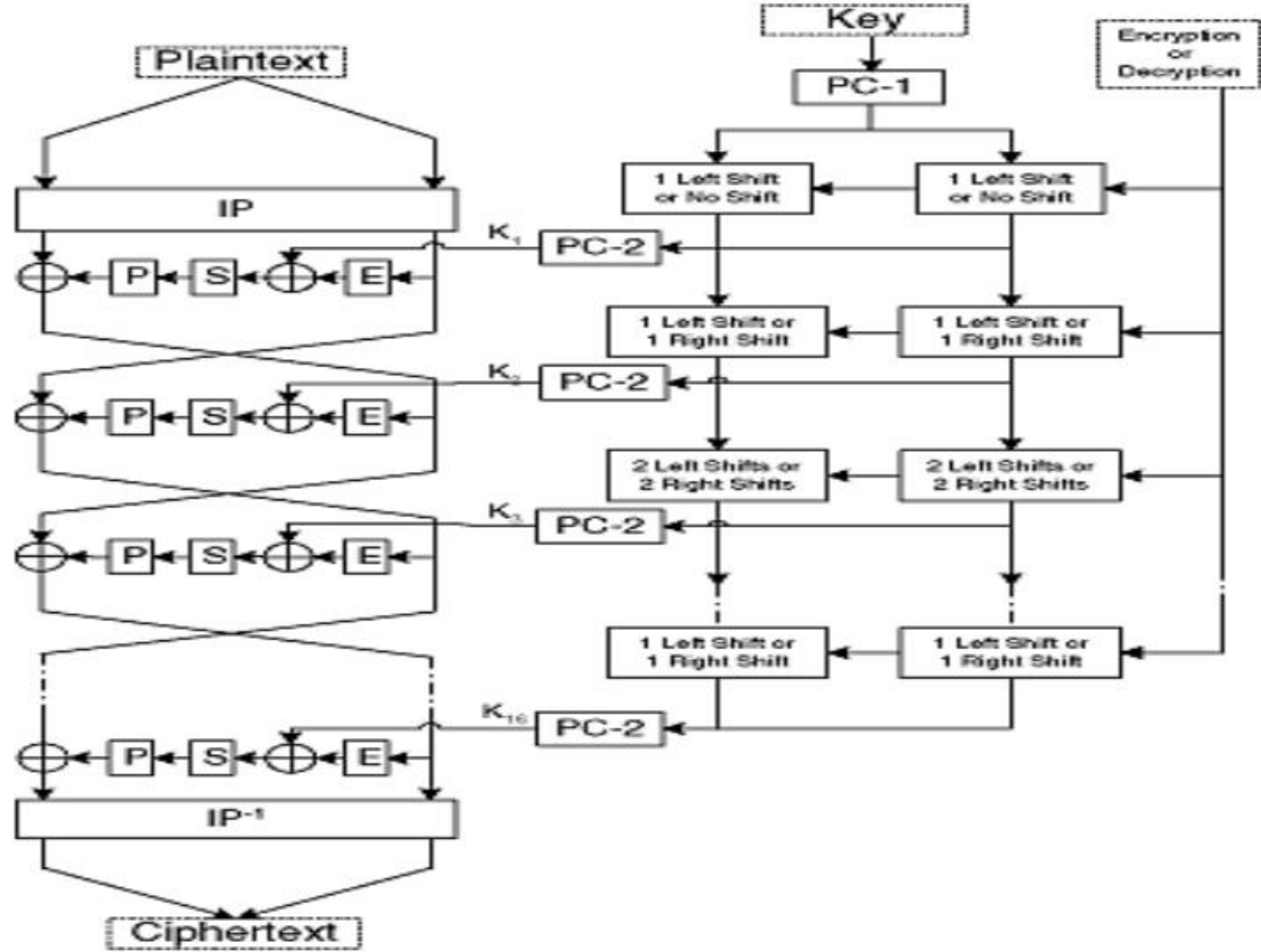
DES轮函数 g 中的 f 函数(续)

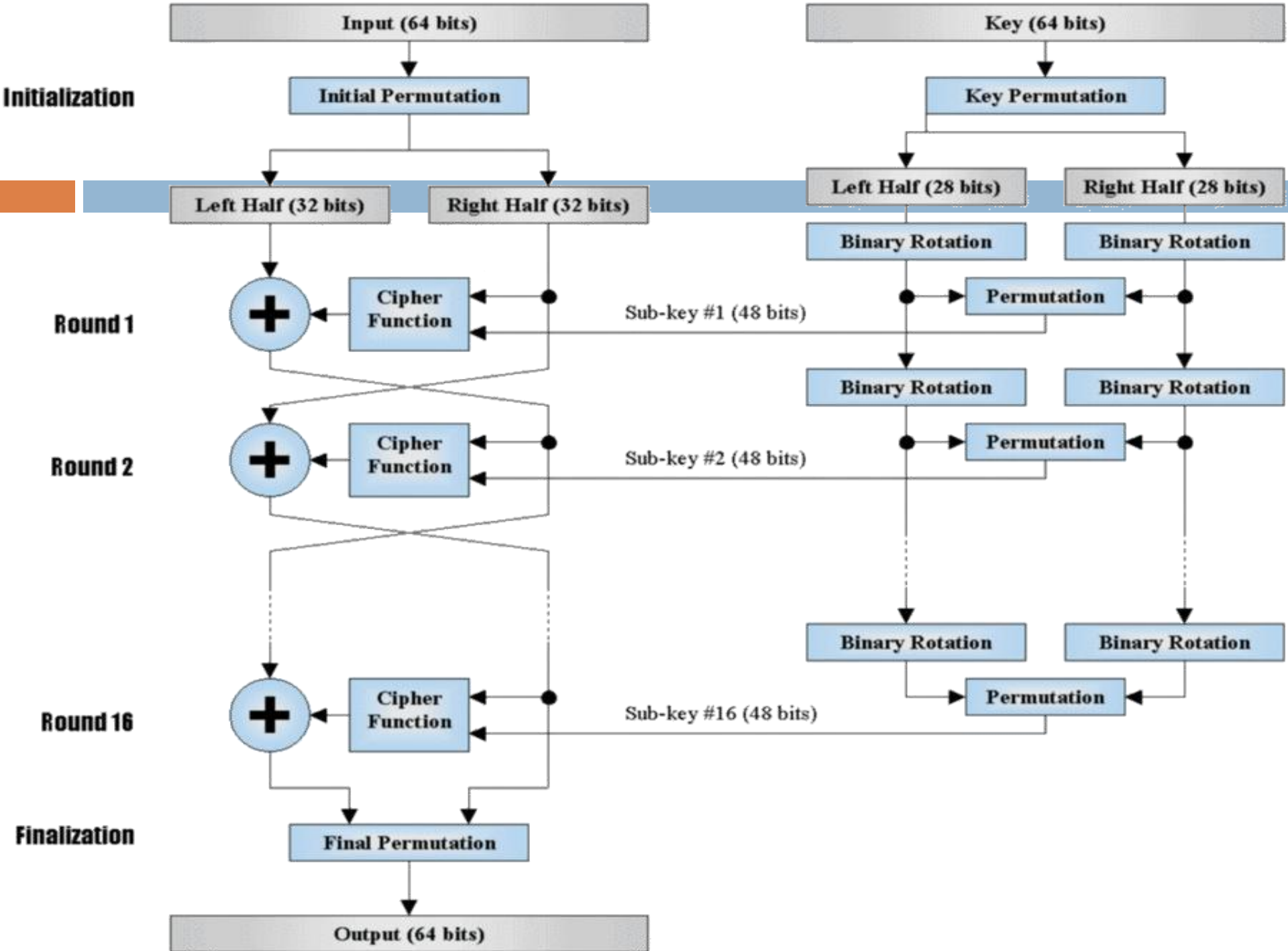
- **扩张**：用扩张置换E将长度为32位的串扩展成为长度为48位的串；
- **密钥混合**：用异或操作将扩张的结果和一个子密钥进行混合；
- **S盒**：在与子密钥混合之后，块被分成8个6位的块，然后使用S盒进行代换处理。8个S盒均以查找表方式提供非线性的代换将6比特的输入变成4比特的输；
- **置换**：S盒的8个输出组合重新组合成长度为32比特的串，然后利用固定的P置换操作，输出结果即为整个 f 函数的输出。



分组密码DES的密钥编排方案

- 首先，使用**选择置换1**从64比特的初始密钥中选出56比特，剩下的8位要么直接丢弃，要么作为奇偶校验位。
- 然后，56比特分成两个28比特的半密钥。每个半密钥接下来都被分别处理。
- 在接下来的轮次中，两个半密钥都被循环左移1或2位（由轮次数决定），然后通过**选择置换2**产生48位的子密钥，每个半密钥24位。图中用<<<表示循环左移。
- 循环左移的位数与轮次的关系可以由数组{ 1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1 }表示。





分组密码AES的组成部分

- 和DES一样，AES也包含轮函数和密钥扩展算法两个部件。但是AES轮函数的结构与DES不同。
- 轮函数由4个部份组成：(1)字节代换(SubBytes); (2)行移变换(ShiftRows); (3)列混合变换(MixColumns); (4)密钥混合(AddRoundKey)
- 密钥扩展算法：将初始密钥扩展成为多个长度相同的子密钥。
- AES轮函数的结构被称为SPN结构。AES是一个特定轮数Nr的SPN型密码。它是以轮函数为循环体的特定轮数Nr的循环操作，每一轮循环需要使用一个由密钥扩展算法生成的子密钥。

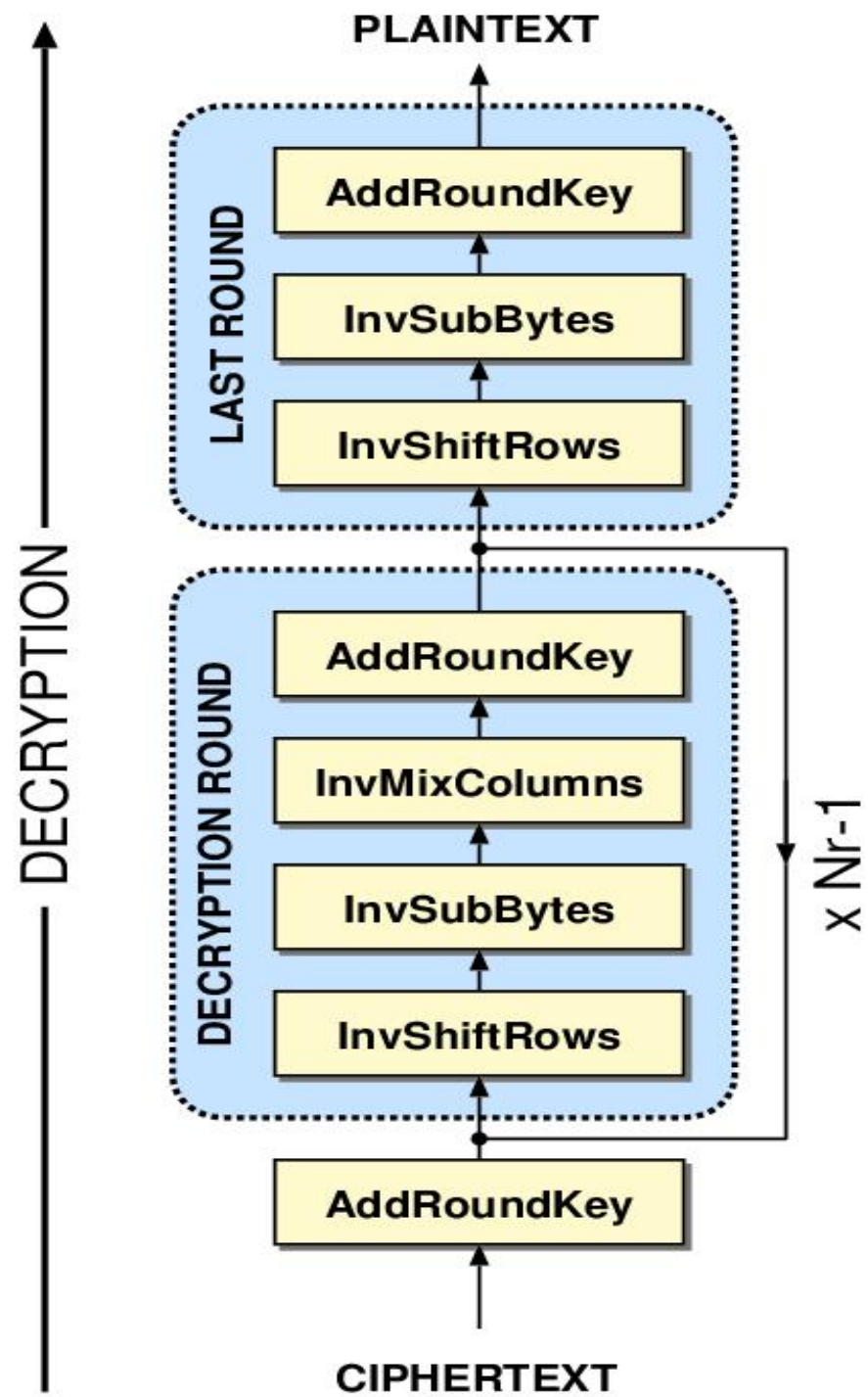
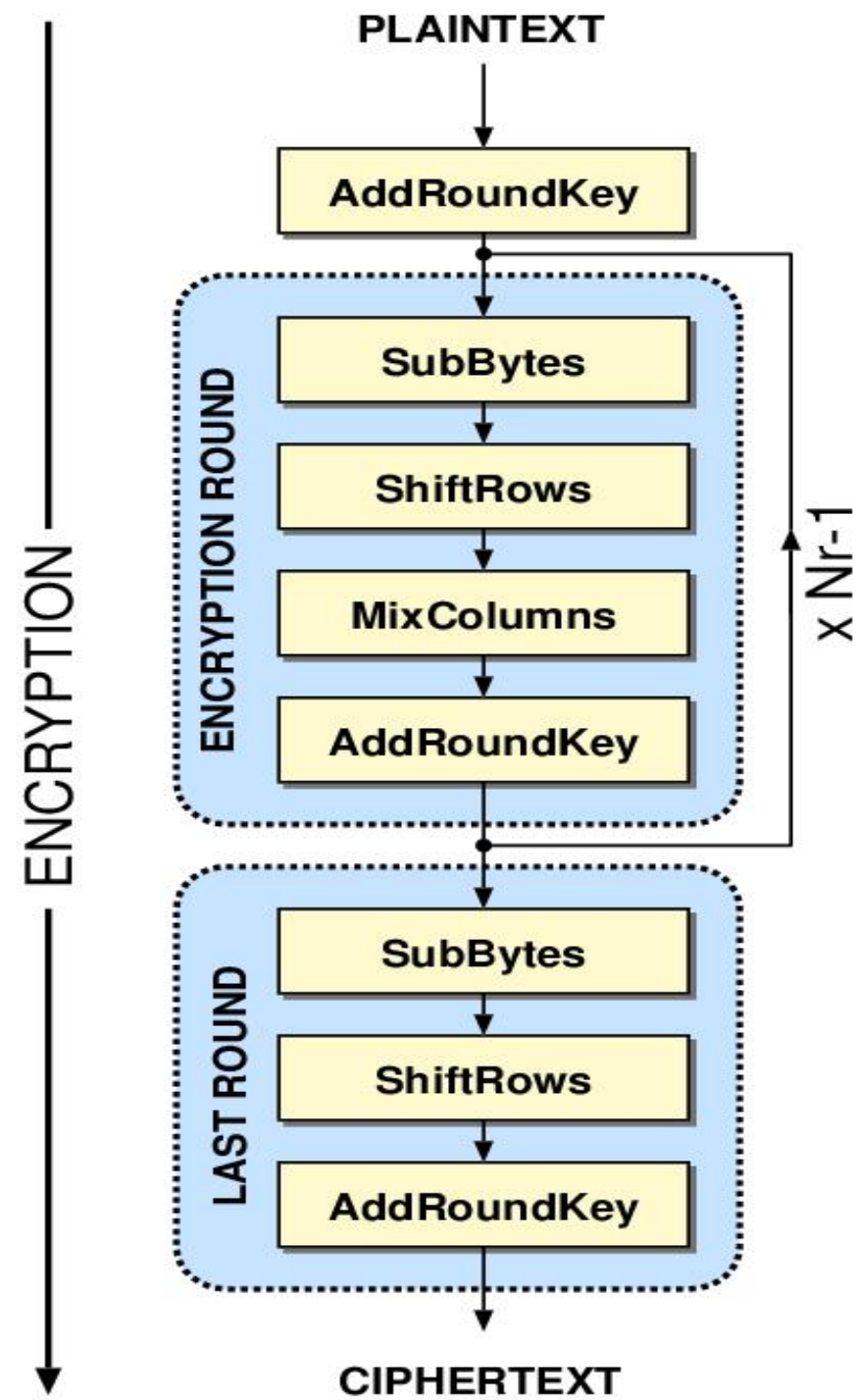
分组密码AES：基本特点

- AES使用的SPN（代换置换网络）结构，不属于DES所使用的Feistel结构。
- AES能有效抵抗当时所有已知的攻击。
- 支持128/192/256位等三种密钥长度。
- 以128比特为一个分组，结构简单，运算速度快。
- 一个128比特的分组再分成16个字节，构成一个4乘4的状态矩阵：

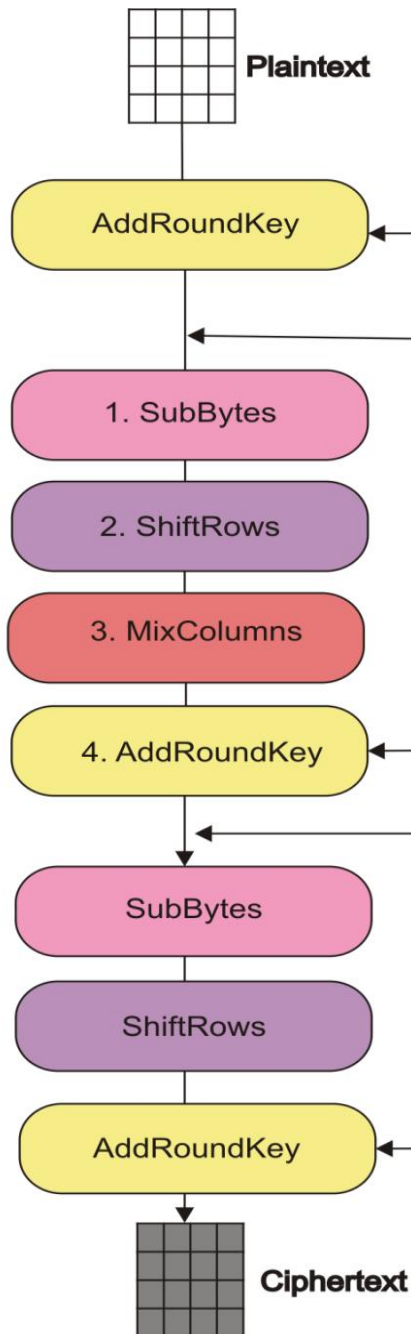
$$a_{0,0}a_{1,0}a_{2,0}a_{3,0}a_{0,1}\cdots\cdots a_{2,3}a_{3,3} = \begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,2} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \end{pmatrix}$$

分组密码AES：总体工作流程

- 128比特的分组经过轮函数的Nr次作用得到128比特的密文分组。
- 128比特的分组在第一次将要被轮函数作用之前，需要与128比特的子密钥按比特异或一次，这一操作被称为AddRoundKey。
- 分组在每一次被轮函数作用时都需要与128比特的子密钥进行一次AddRoundKey操作。
- 需要Nr个不同的子密钥做Nr次AddRoundKey操作，加上第一次的AddRoundKey操作，密钥扩展算法须将初始密钥扩展成为Nr+1个长度子密钥。



1st Round

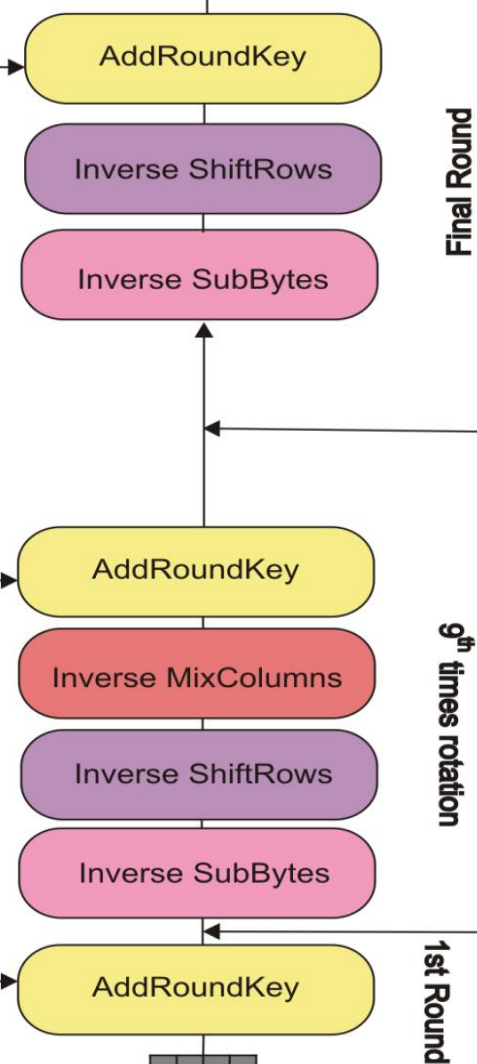


Key

Expansion Key

Plaintext

Ciphertext



Final Round

9th times rotation

1st Round

D_1 D_2 D_3 D_4 D_5 D_6 D_7 D_8 D_9 D_{10} D_{11} D_{12} D_{13} D_{14} D_{15} D_{16}

D_1	D_5	D_9	D_{13}
D_2	D_6	D_{10}	D_{14}
D_3	D_7	D_{11}	D_{15}
D_4	D_8	D_{12}	D_{16}

K_1 K_2 K_3 K_4 K_5 K_6 K_7 K_8 K_9 K_{10} K_{11} K_{12} K_{13} K_{14} K_{15} K_{16}

K_1	K_5	K_9	K_{13}
K_2	K_6	K_{10}	K_{14}
K_3	K_7	K_{11}	K_{15}
K_4	K_8	K_{12}	K_{16}

KeyExpansion

R_0	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
R_1	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
R_2	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
R_3	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
R_4	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
R_5	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
R_6	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
R_7	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
R_8	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
R_9	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
R_{10}	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.

Round 0

AddRoundKey

Round 1

SubBytes

ShiftRows

MixColumns

Round 9

AddRoundKey

Round 10

SubBytes

ShiftRows

AddRoundKey

C_1 C_2 C_3 C_4 C_5 C_6 C_7 C_8 C_9 C_{10} C_{11} C_{12} C_{13} C_{14} C_{15} C_{16}

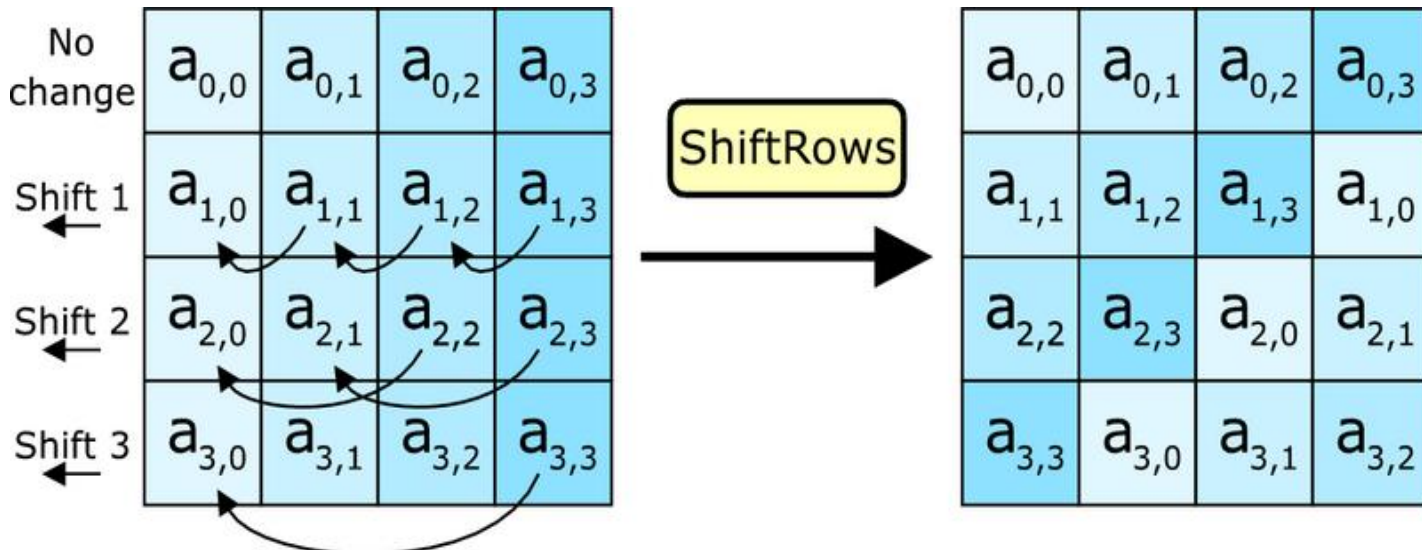
字节代换 (*SubBytes*)

- **SubBytes**是一种分别作用在单个字节上的非线性变换，由16次**SubByte**运算组成。
- **SubByte**由两部分变换合成：
 - ① 把输入的字节看成有限域 $GF(2^8)$ 中的元素，求其有限域 $GF(2^8)$ 中的乘法逆元素。有限域 $GF(2^8)$ 中的元素是8维的二元向量，或者说是代数次数至多为7的一元多项式。
 - ② 对得到的乘法逆元素进行固定的仿射变换
- 给定一个字节，**SubByte**输出一个唯一与之对应的字节。对于所有可能的字节，事先计算其**SubByte**输出值，可以构造一个256行256列的表，这个表即为AES的S-盒。

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

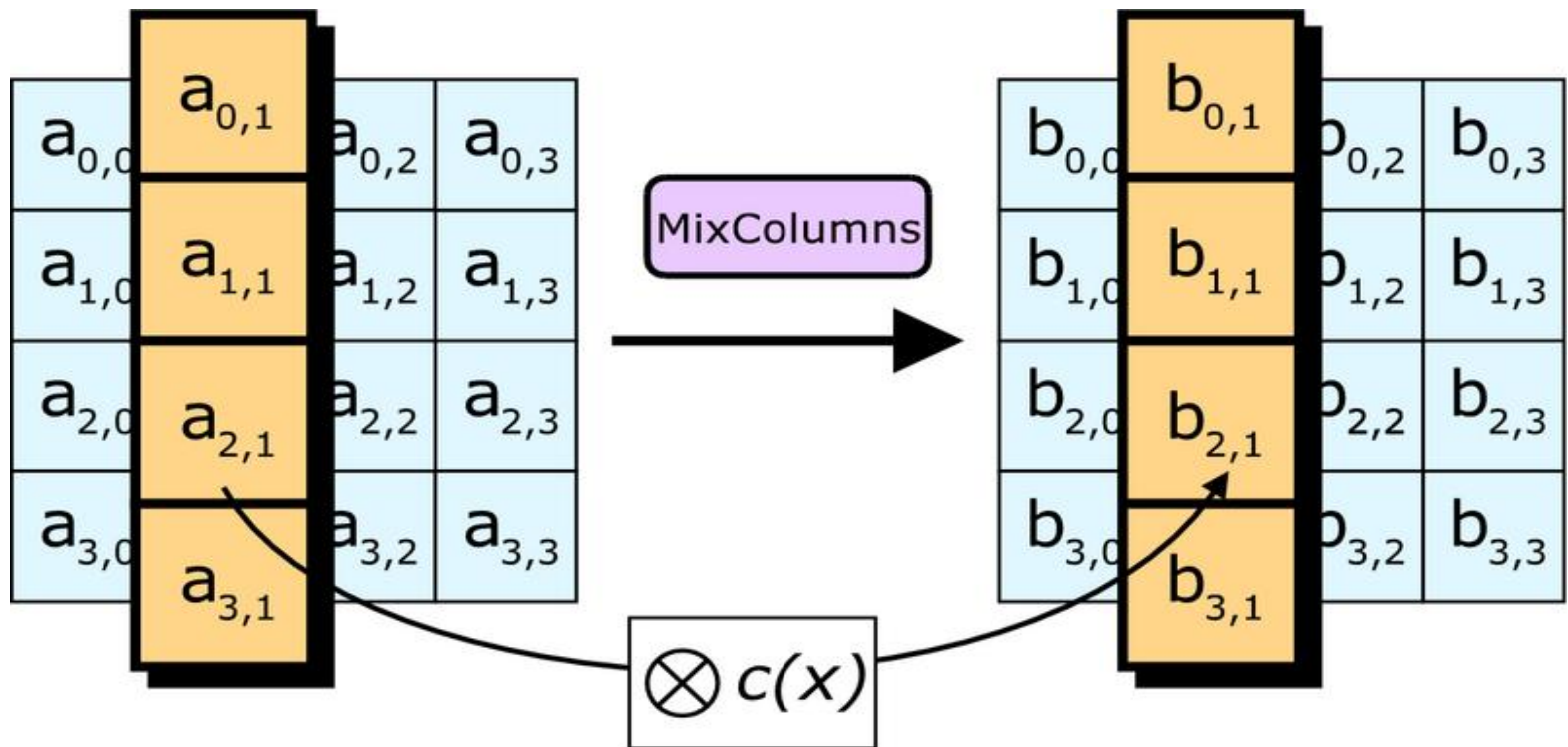
行移变换

- 行移变换 (ShiftRows) 分别对状态矩阵的每一行进行循环移位操作
 - ① 第0行保持不变
 - ② 第1行循环左移1个字节 (8比特)
 - ③ 第2行循环左移2个字节 (16比特)
 - ④ 第3行循环左移3个字节 (24比特)



列混合变换

- 列混合变换 (MixColumns) 用固定的4乘4矩阵分别乘以状态矩阵的每一列，即分别对状态矩阵的每一列进行希尔(Hill)密码操作。



行移变换和列混合变换

- 在子密钥的参与下，字节代换SubBytes在SPN结构中起到混淆作用。
- 行移变换ShiftRows和列混合变换MixColumns在SPN结构中起到扩散作用。
- ✓ 混淆：明文、密文和密钥之间关系十分复杂，难以数学描述，或从统计上分析。
- ✓ 扩散：明文的任意一比特对密文的所有比特都有影响。

AES的密钥扩展

- AES的密钥扩展算法产生 $Nb(Nr+1)$ 个字,最初的 Nk 个字为初始密钥,后面的轮密钥根据前面的字递归得到。

	密钥字长 Nk	分组长度 Nb	迭代轮数 Nr
AES-128	4	4	10
AES-192	6	4	12
AES-256	8	4	14

AES的密钥扩展算法

KeyExpansion

- 密钥扩展算法包括RotWord和SubWord两种操作，以及一共10个字的常数数组

$Rcon[1], \dots, Rcon[10]$

- RotWord操作对四个字节进行循环移位操作，

$RotWord(B_0, B_1, B_2, B_3) = (B_1, B_2, B_3, B_0)$

- SubWord操作对四个字节分别用S盒进行字节代换，即

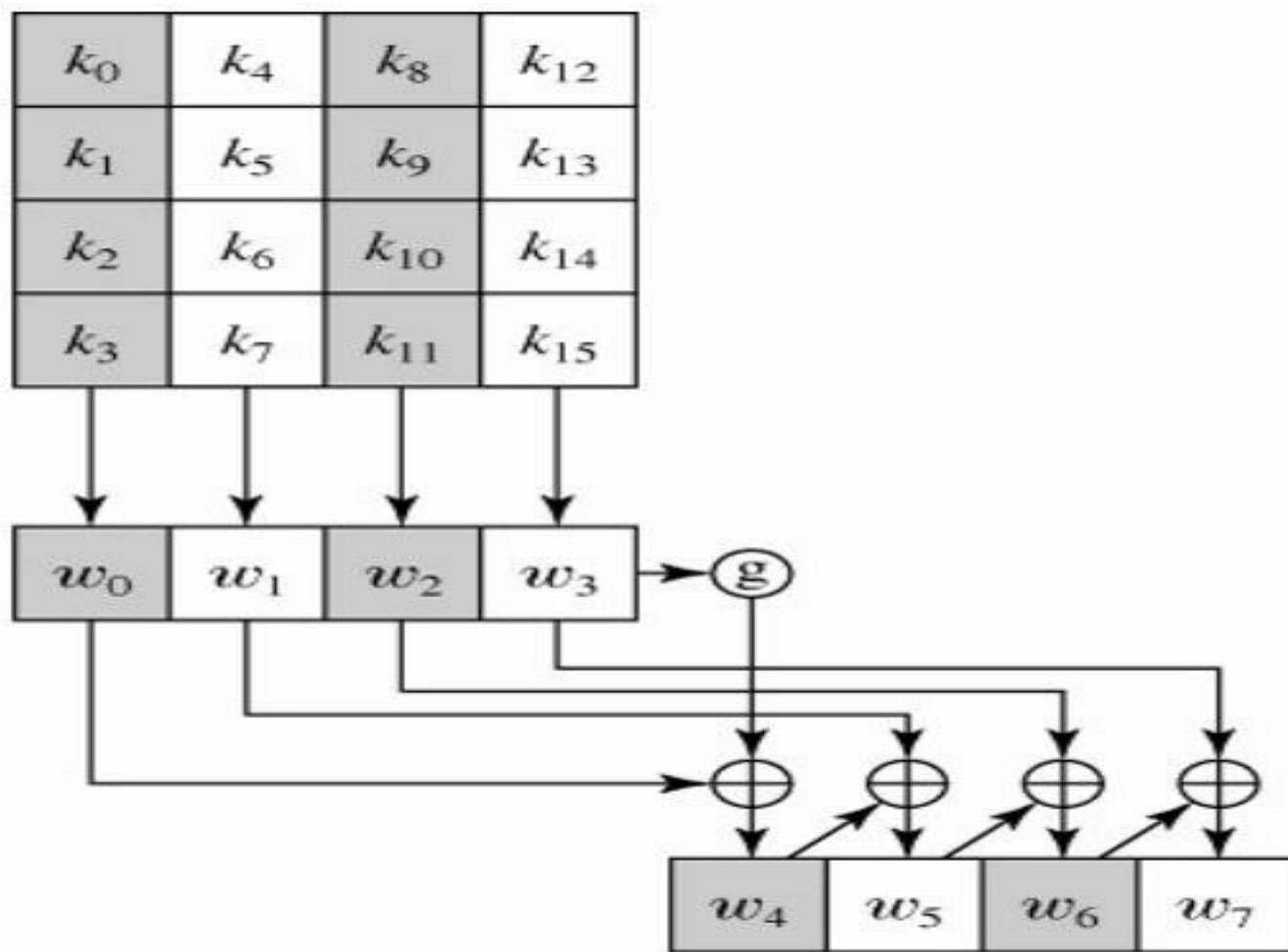
$SubWord(B_0, B_1, B_2, B_3) = (B_0', B_1', B_2', B_3')$

其中 $B_i' = SubBytes(B_i)$ 。

AES的密钥扩展算法(续)

```
for  $i \leftarrow 0$  to 3
  do  $w[i] \leftarrow (key[4i], key[4i + 1], key[4i + 2], key[4i + 3])$ 
for  $i \leftarrow 4$  to 43
  do  $\begin{cases} temp \leftarrow w[i - 1] \\ \text{if } i \equiv 0 \pmod{4} \\ \quad \text{then } temp \leftarrow \text{SUBWORD}(\text{ROTWORD}(temp)) \oplus RCon[i/4] \\ w[i] \leftarrow w[i - 4] \oplus temp \end{cases}$ 
return  $(w[0], \dots, w[43])$ 
```

AES的密钥扩展算法(续)



AES的解密过程

- AES的解密过程与AES的解密过程时类似的。
- SubBytes、ShiftRows和MixColumns有相应的逆变换，即InvSubBytes、InvShiftRows和InvMixColumns。
- 轮函数中由从SubBytes到ShiftRows再到MixColumns的顺序变为从InvMixColumns到InvShiftRows再到InvSubBytes的顺序。
- 关于AES想了解更多？建议阅读FIPS-PUB-197标准：<http://csrc.nist.gov/publications/fips/fips197/fips-197.pdf>

混淆和扩散

- 分组密码的设计中用代替（代换）和置换实现**混淆**（Confusion）和**扩散**（Diffusion）功能。
- 混淆：明文、密文和密钥之间关系十分复杂，难以数学描述，或从统计上分析
- 扩散：明文的任意一比特对密文的所有比特都有影响
- 一般的分组密码体制都包含**轮函数**和**密钥扩展算法**两个部件。

分组密码的迭代次数

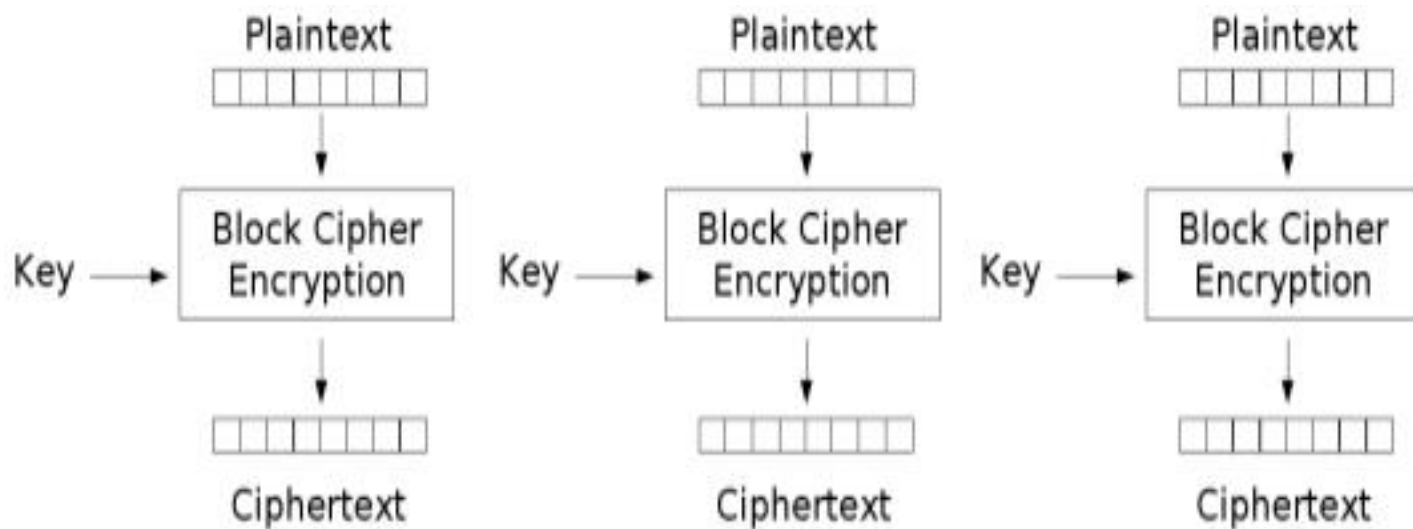
- 在一般的分组密码中，一个明文分组经过轮函数的若干次作用得到一个密文分组。
- 一个明文分组经过轮函数的作用次数通常称为**迭代次数**。
- 一般地，迭代次数越大，安全性越高，但效率越低。
- 通常，一轮迭代操作前、或迭代操作后、或迭代操作中需要一个子密钥的参加。
- **DES**的迭代次数为**16**，**AES**的迭代次数与密钥长度有关，分别为**10次**，**12次**和**14次**。

分组密码的工作模式

- 密码学家针对**DES**开发了四种工作模式（使用方式）。**AES**继承了**DES**的这四种工作模式，并加入了**CTR**模式。实际上，这些工作模式可以应用于任何分组密码。
- ① 电码本（**ECB**, Electronic CodeBook）模式
- ② 密码分组链接（**CBC**, Cipher Block Chaining）模式
- ③ 密码反馈（**CFB**, Cipher FeedBack）模式
- ④ 输出反馈模式（**OFB**, Output FeedBack）模式
- ⑤ 计数器（**CTR**, CounTeR）模式

分组密码之ECB模式

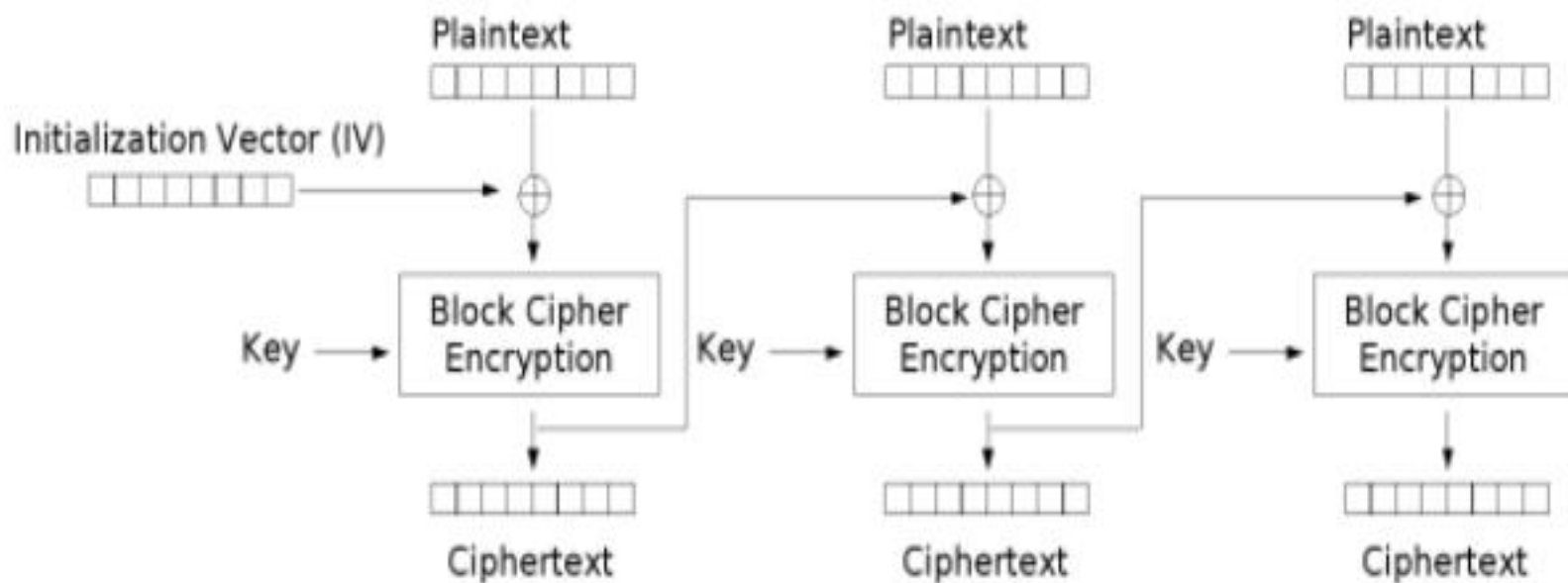
- ECB模式将需要加密的消息按照分组长度被分为数个块，并对每个块进行独立加密。它的缺点是同样的明文块会被加密成相同的密文块，因此不能很好地保护数据，所以也并不推荐使用。



Electronic Codebook (ECB) mode encryption

分组密码之CBC模式

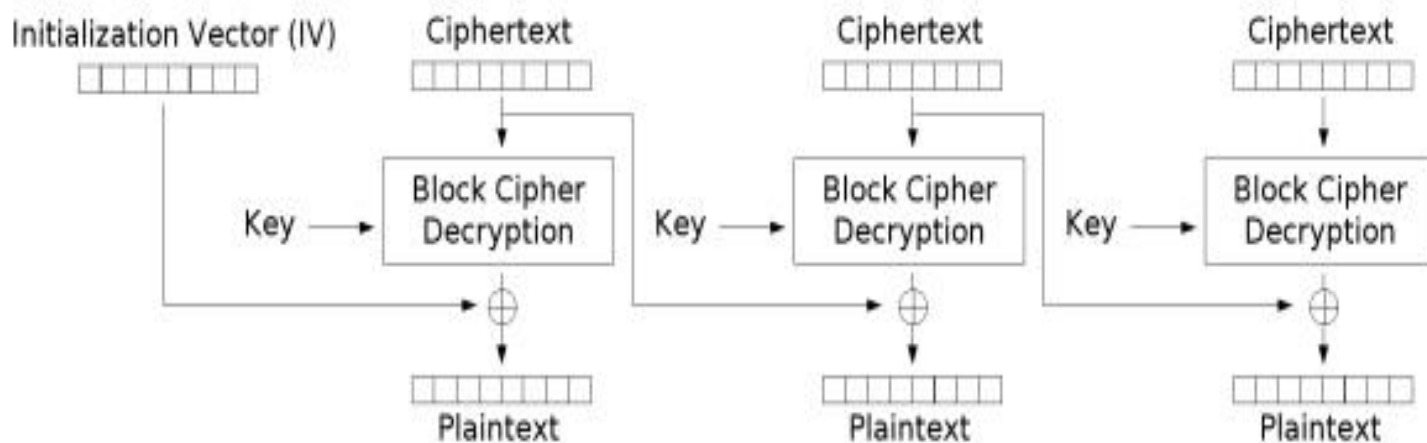
- 在CBC模式中，每个明文块先与前一个密文块进行异或，然后再进行加密。加密后的每个密文分组都依赖于它前面的所有明文分组。加密过程串行，密文分组被篡改影响后续一个分组解密。



Cipher Block Chaining (CBC) mode encryption

分组密码之CBC模式(续)

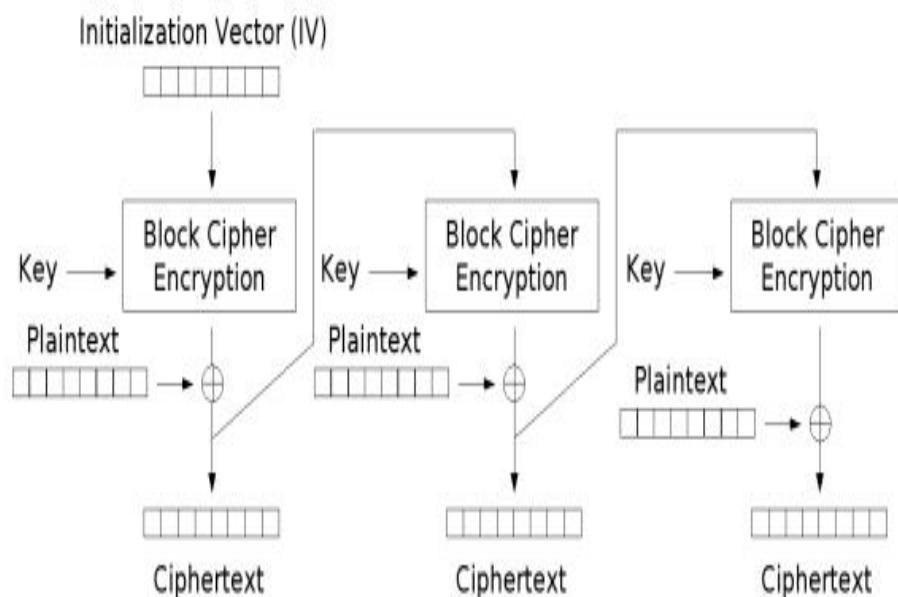
- CBC模式加密过程无法并行化，解密可以并行化。密文中一比特的改变会导致当前明文分组和下一个明文分组发生改变，但不会影响到其他密文分组的解密。此外，CBC模式的一个缺陷是能为敌手开展选择明文攻击提供某些帮助。



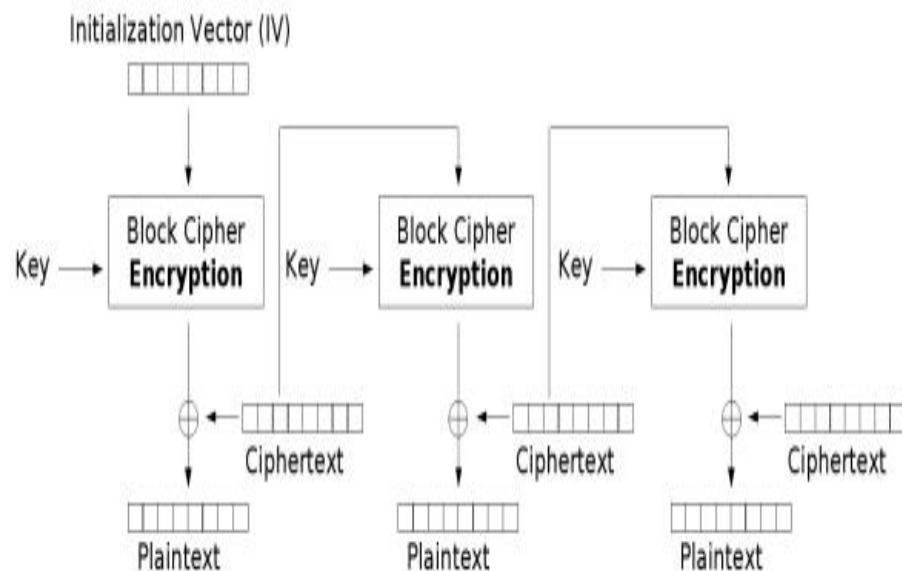
Cipher Block Chaining (CBC) mode decryption

分组密码之CFB模式

- CFB模式将分组密码变为了自同步流密码。与CBC模式相似，加密过程不能并行化，而解密过程可以并行化，密文分组被篡改影响后续的所有密文分组的解密。



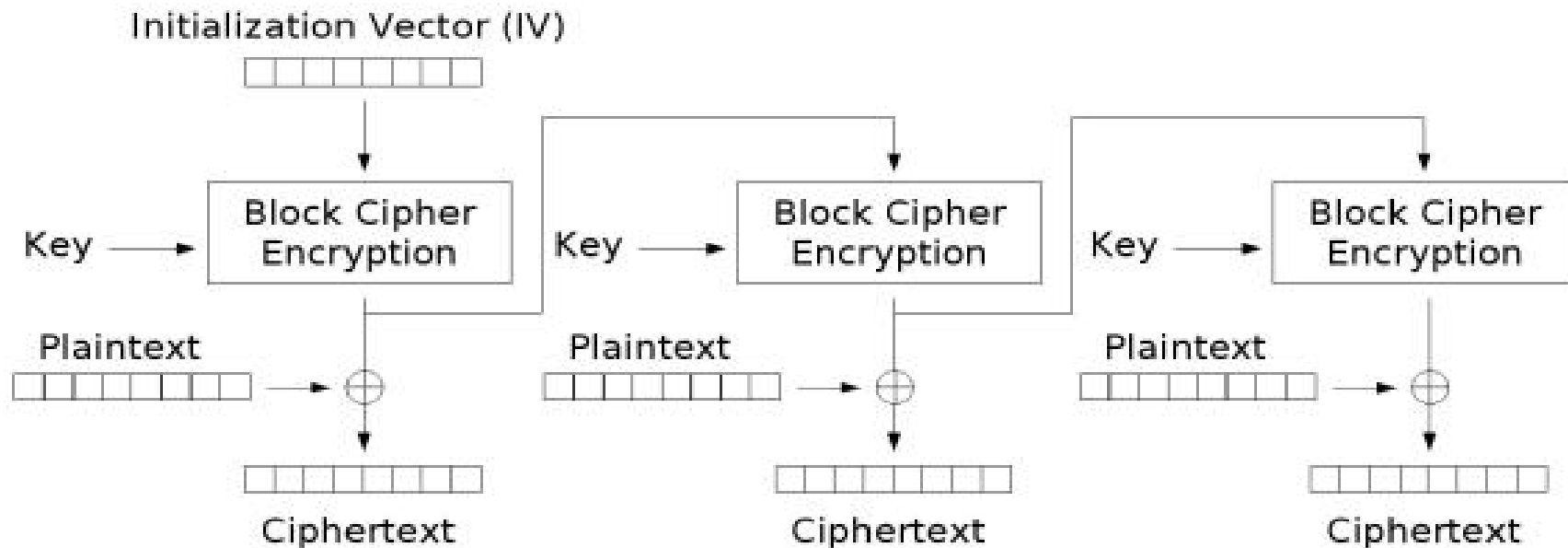
Cipher Feedback (CFB) mode encryption



Cipher Feedback (CFB) mode decryption

分组密码之OFB模式

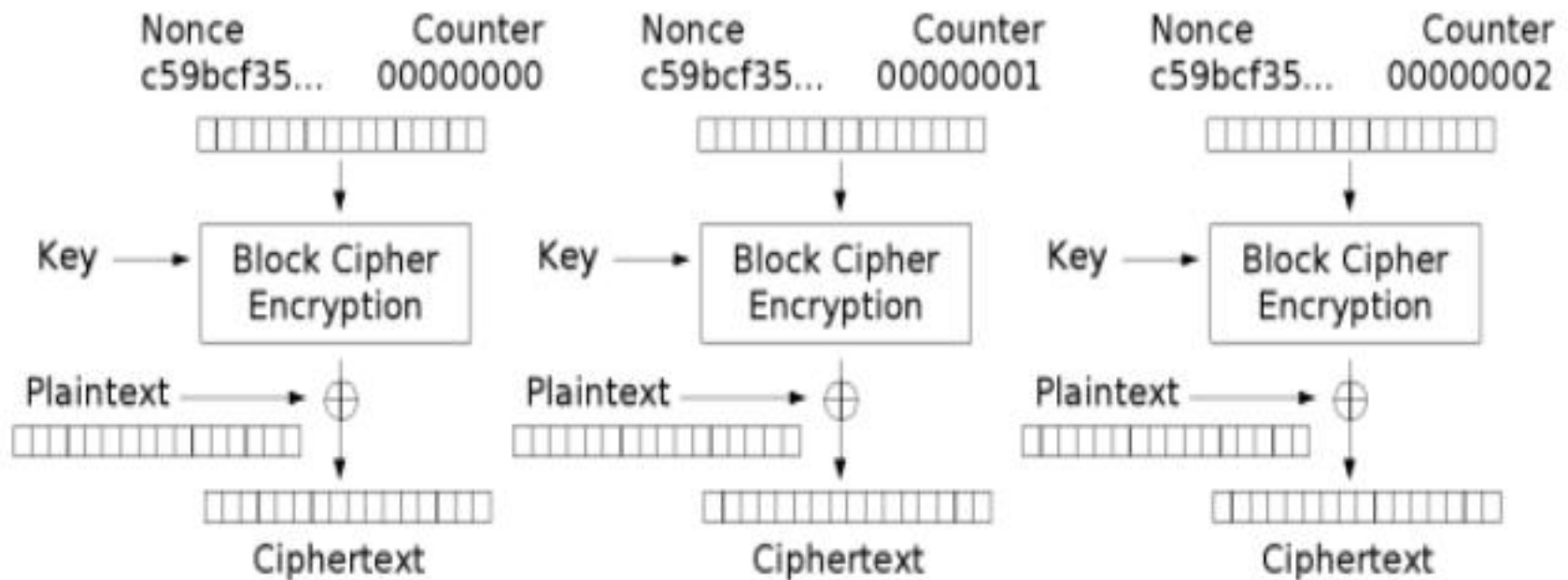
- OFB模式也是将分组密码变成了同步流密码。特点是加密和解密过程都不能并行化，前面明文分组的加密结果与后面的无关。密文被篡改仅影响对应位置的解密结果。



Output Feedback (OFB) mode encryption

分组密码之CTR模式

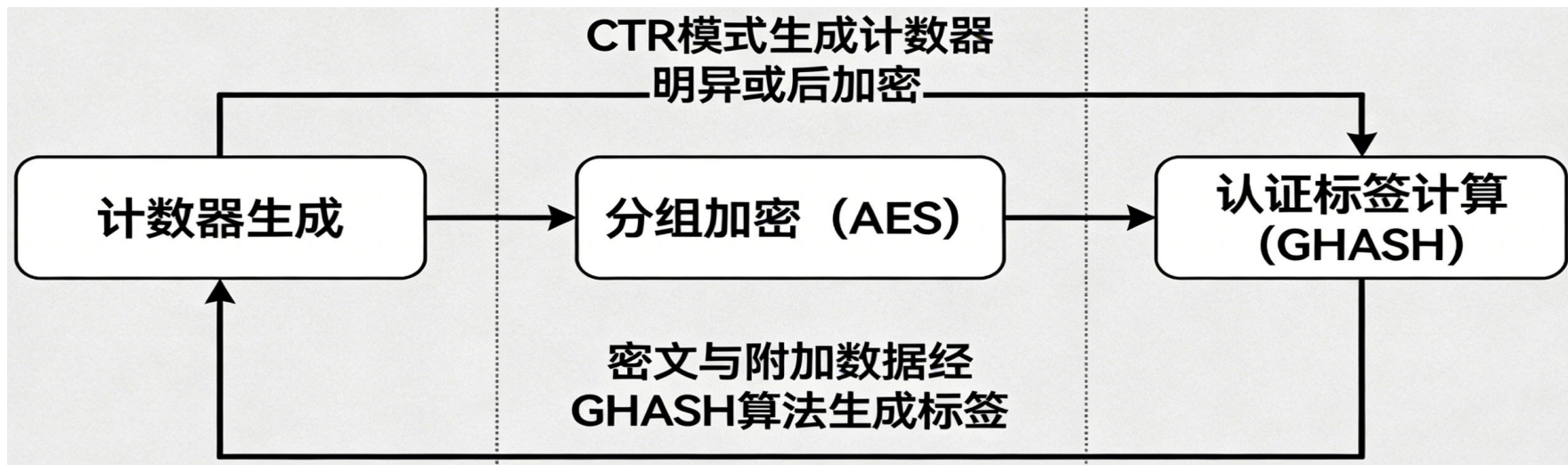
- CTR模式还是将分组密码变成了同步流密码。由于加密和解密过程均可以进行并行处理，CTR适合运用于多处理器的硬件上。



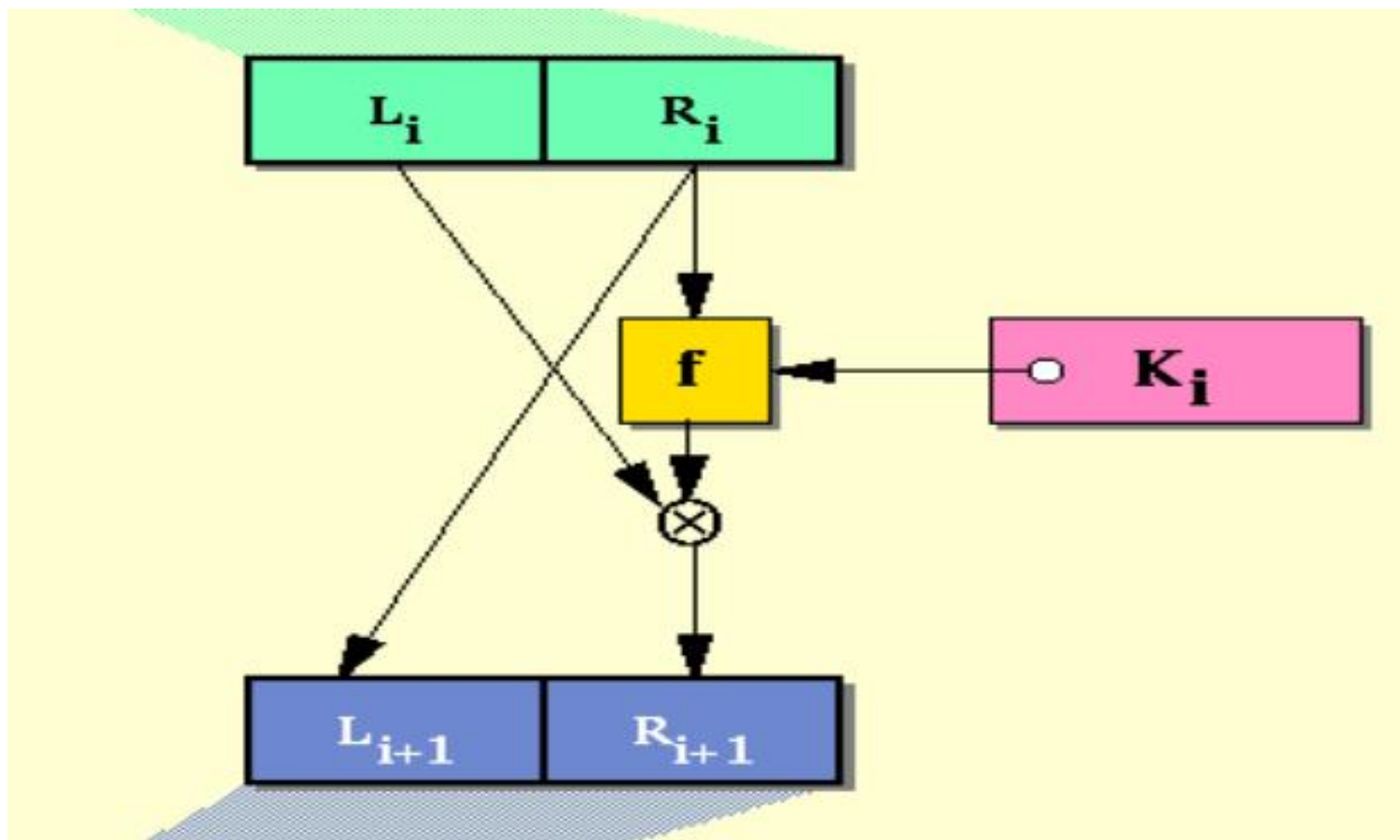
Counter (CTR) mode encryption

分组密码的认证加密工作模式

- 支持认证加密的分组密码工作模式（简称 AE 模式），是将数据机密性保护与完整性、不可篡改性认证一体化设计的分组密码应用方案。
- 典型代表有 GCM、CCM、EAX 等工作模式，大多基于 CTR 等高效加密模块结合专用认证算法构建，兼具并行处理能力和灵活的关联数据（AAD）认证功能。



DES使用的Feistel密码结构



程序设计语言中的逻辑位运算

运算符

□ &

□ |

□ ^

□ !

□ <<

□ >>

功能

逻辑与

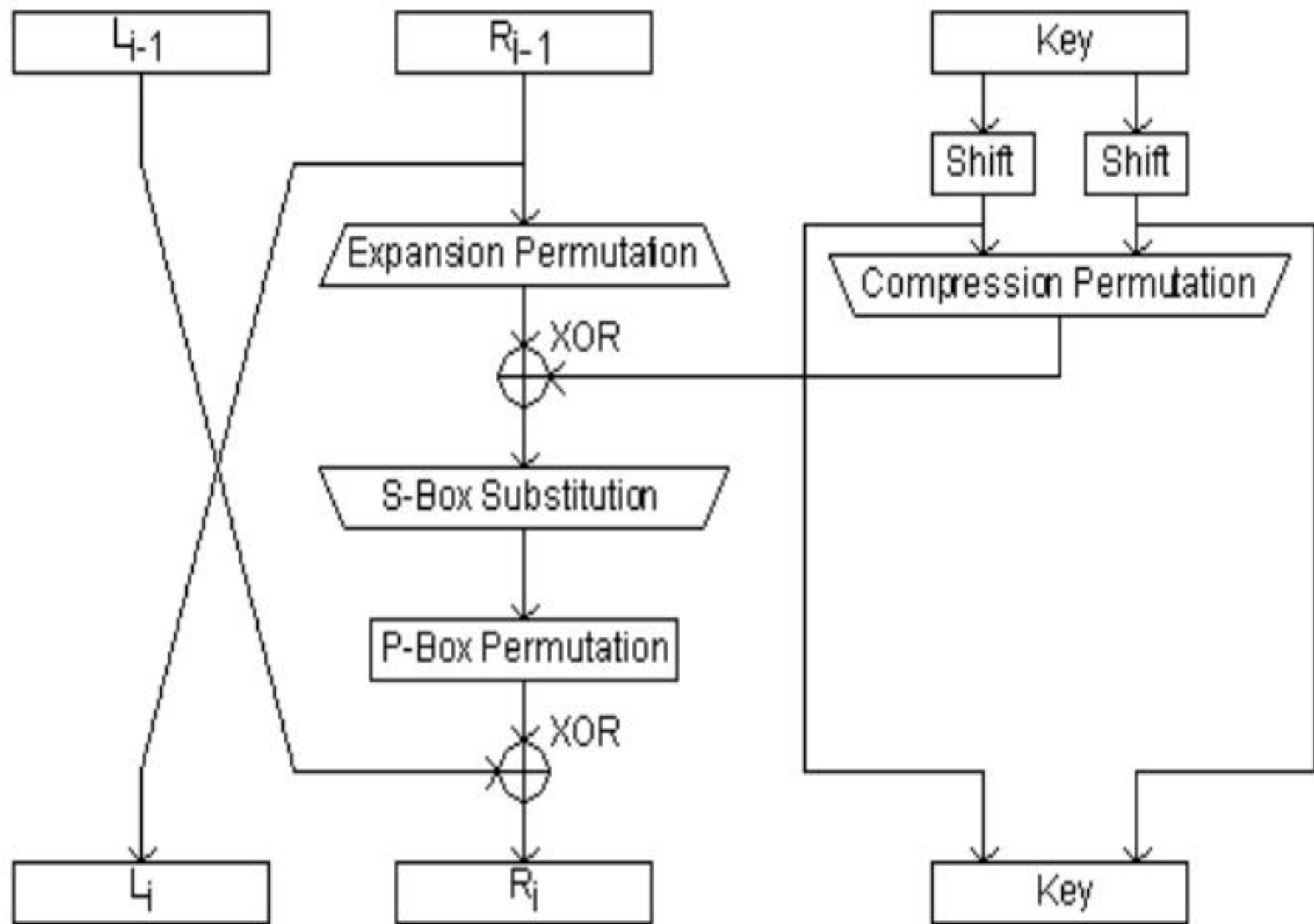
逻辑或

逻辑异或

逻辑非

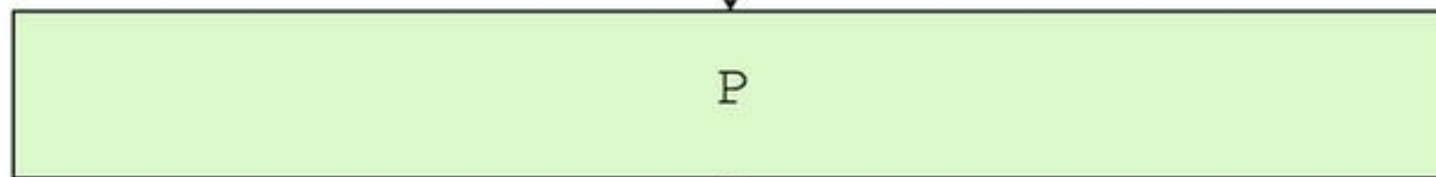
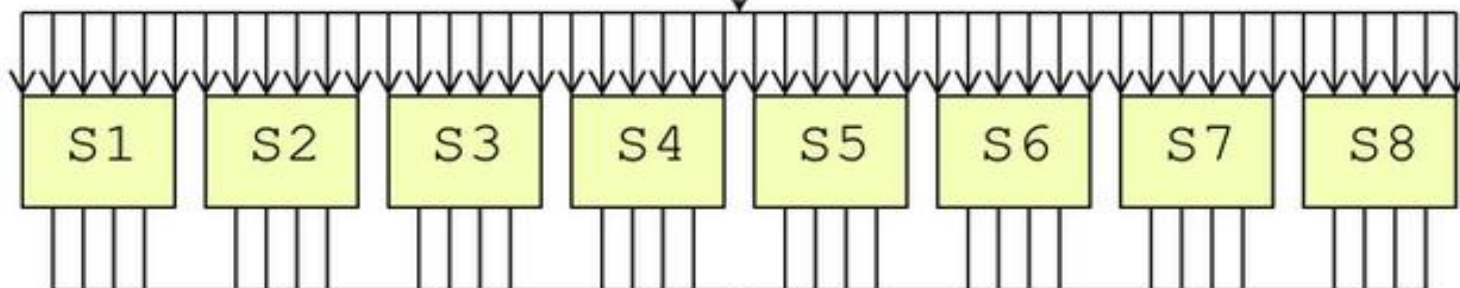
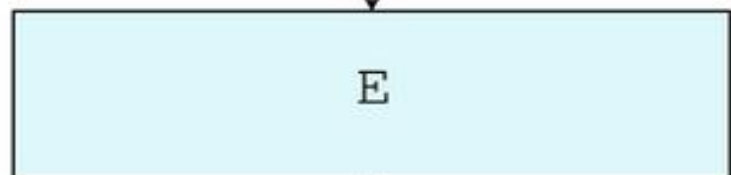
按位左移

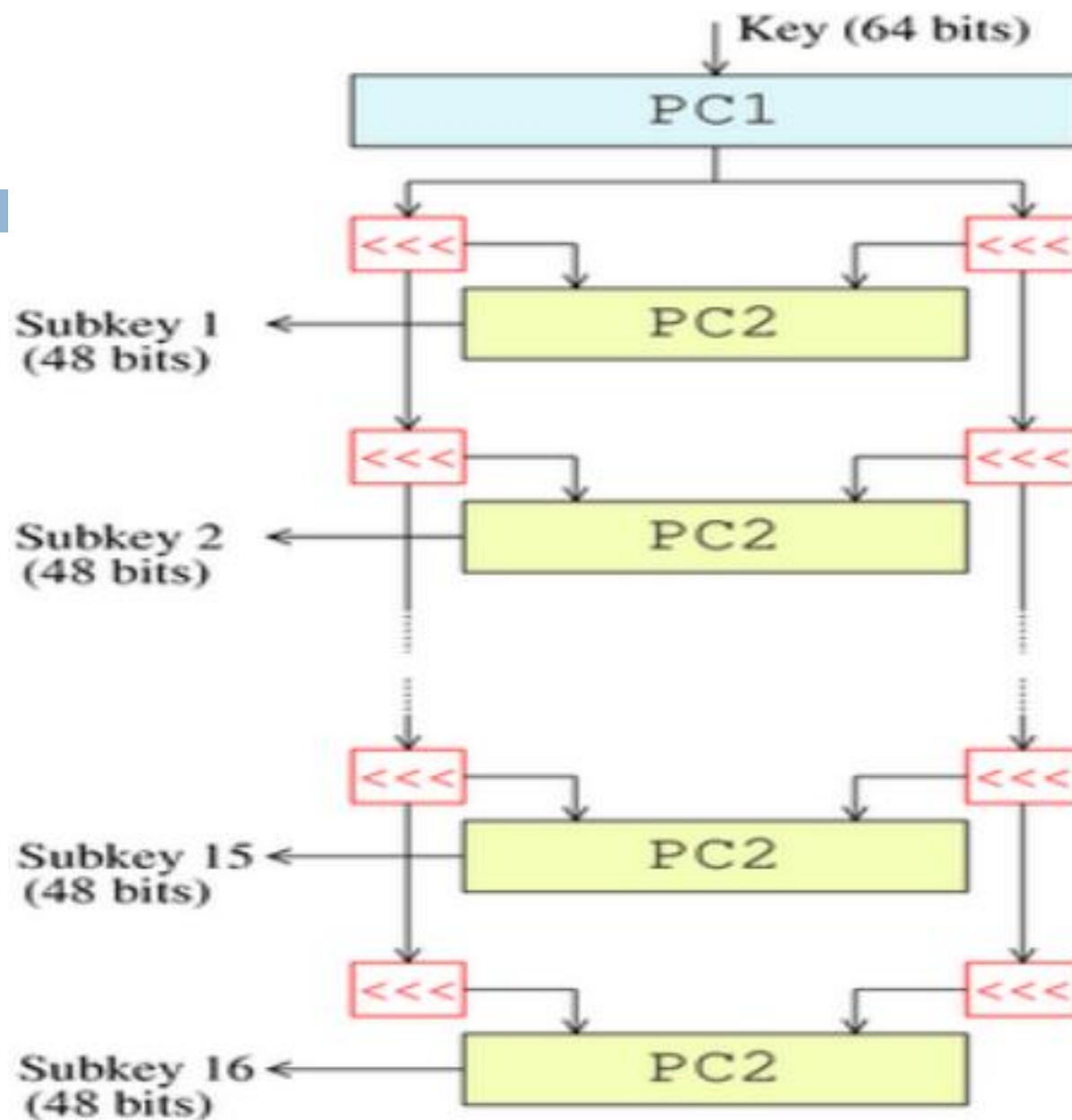
按位右移

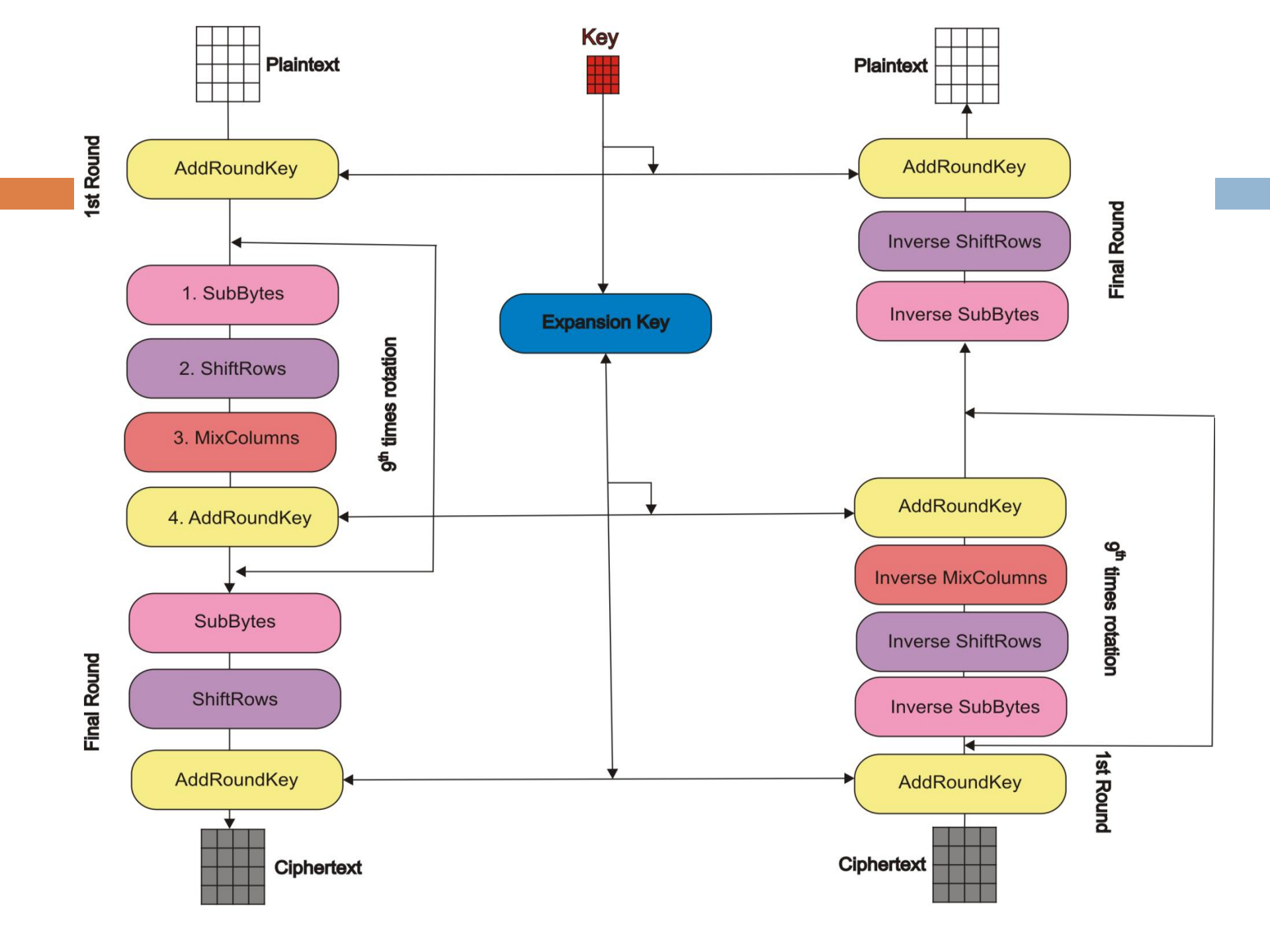


Half Block (32 bits)

Subkey (48 bits)







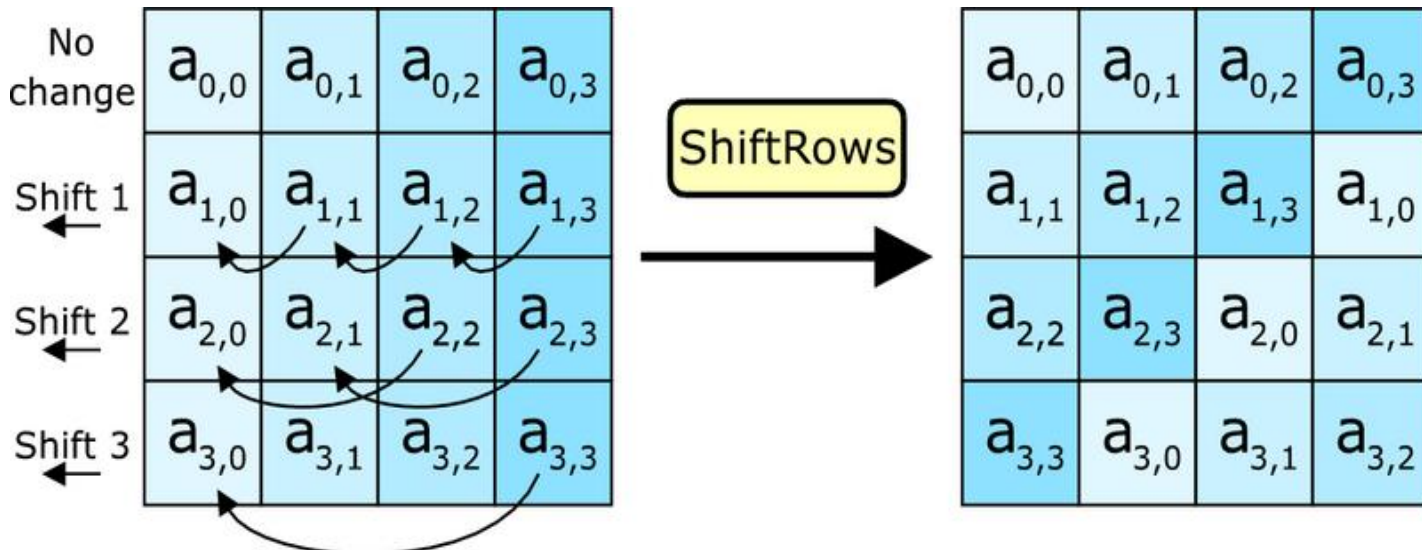
字节代换 (*SubBytes*)

- **SubBytes**是一种分别作用在单个字节上的非线性变换，由16次**SubByte**运算组成。
- **SubByte**由两部分变换合成：
 - ① 把输入的字节看成有限域 $GF(2^8)$ 中的元素，求其有限域 $GF(2^8)$ 中的乘法逆元素
 - ② 对得到的乘法逆元素进行固定的仿射变换
- 给定一个字节，**SubByte**输出一个唯一与之对应的字节。对于所有可能的字节，事先计算其**SubByte**输出值，可以构造一个16行16列的表，这个表即为**AES**的**S-盒**。

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

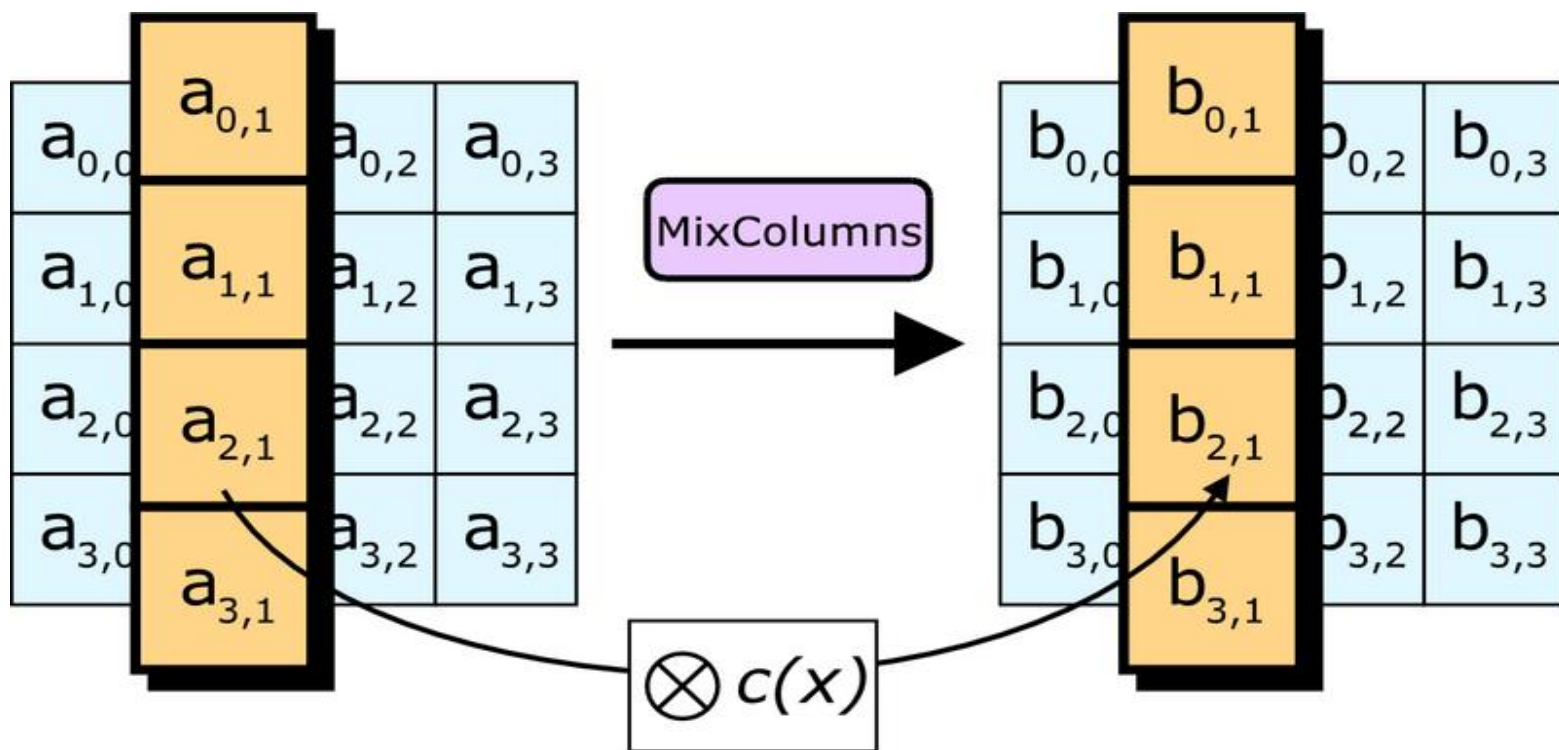
行移变换

- 行移变换 (ShiftRows) 分别对状态矩阵的每一行进行循环移位操作
 - ① 第0行保持不变
 - ② 第1行循环左移1个字节 (8比特)
 - ③ 第2行循环左移2个字节 (16比特)
 - ④ 第3行循环左移3个字节 (24比特)



列混合变换

- 列混合变换 (MixColumns) 用固定的4乘4矩阵分别乘以状态矩阵的每一列，即分别对状态矩阵的每一列进行希尔(Hill)密码操作。



MixColumns模乘表

```
byte Mul02_table[256]={  
0x0, 0x2, 0x4, 0x6, 0x8, 0xa, 0xc, 0xe, 0x10, 0x12, 0x14, 0x16, 0x18,  
0x1a, 0x1c, 0x1e, 0x20, 0x22, 0x24, 0x26, 0x28, 0x2a, 0x2c, 0x2e,  
0x30, 0x32, 0x34, 0x36, 0x38, 0x3a, 0x3c, 0x3e, 0x40, 0x42, 0x44,  
0x46, 0x48, 0x4a, 0x4c, 0x4e, 0x50, 0x52, 0x54, 0x56, 0x58, 0x5a,  
0x5c, 0x5e, 0x60, 0x62, 0x64, 0x66, 0x68, 0x6a, 0x6c, 0x6e, 0x70,  
0x72, 0x74, 0x76, 0x78, 0x7a, 0x7c, 0x7e, 0x80, 0x82, 0x84, 0x86,  
0x88, 0x8a, 0x8c, 0x8e, 0x90, 0x92, 0x94, 0x96, 0x98, 0x9a, 0x9c,  
0x9e, 0xa0, 0xa2, 0xa4, 0xa6, 0xa8, 0xaa, 0xac, 0xae, 0xb0, 0xb2,  
0xb4, 0xb6, 0xb8, 0xba, 0xbc, 0xbe, .....
```

MixColumns操作

```
void MixColumns(byte state[]){  
    byte s[4];  
    for(int i=0;i<16;i=i+4){  
        s[0]=state[i]; s[1]=state[i+1];  
        s[2]=state[i+2]; s[3]=state[i+3];  
        state[i]=Mul02_table[s[0]]^Mul03_table[s[1]]^s[2]^s[3];  
        state[i+1]=s[0]^Mul02_table[s[1]]^Mul03_table[s[2]]^s[3];  
        state[i+2]=s[0]^s[1]^Mul02_table[s[2]]^Mul03_table[s[3]];  
        state[i+3]=Mul03_table[s[0]]^s[1]^s[2]^Mul02_table[s[3]];  
    }//使用模乘表  
}
```

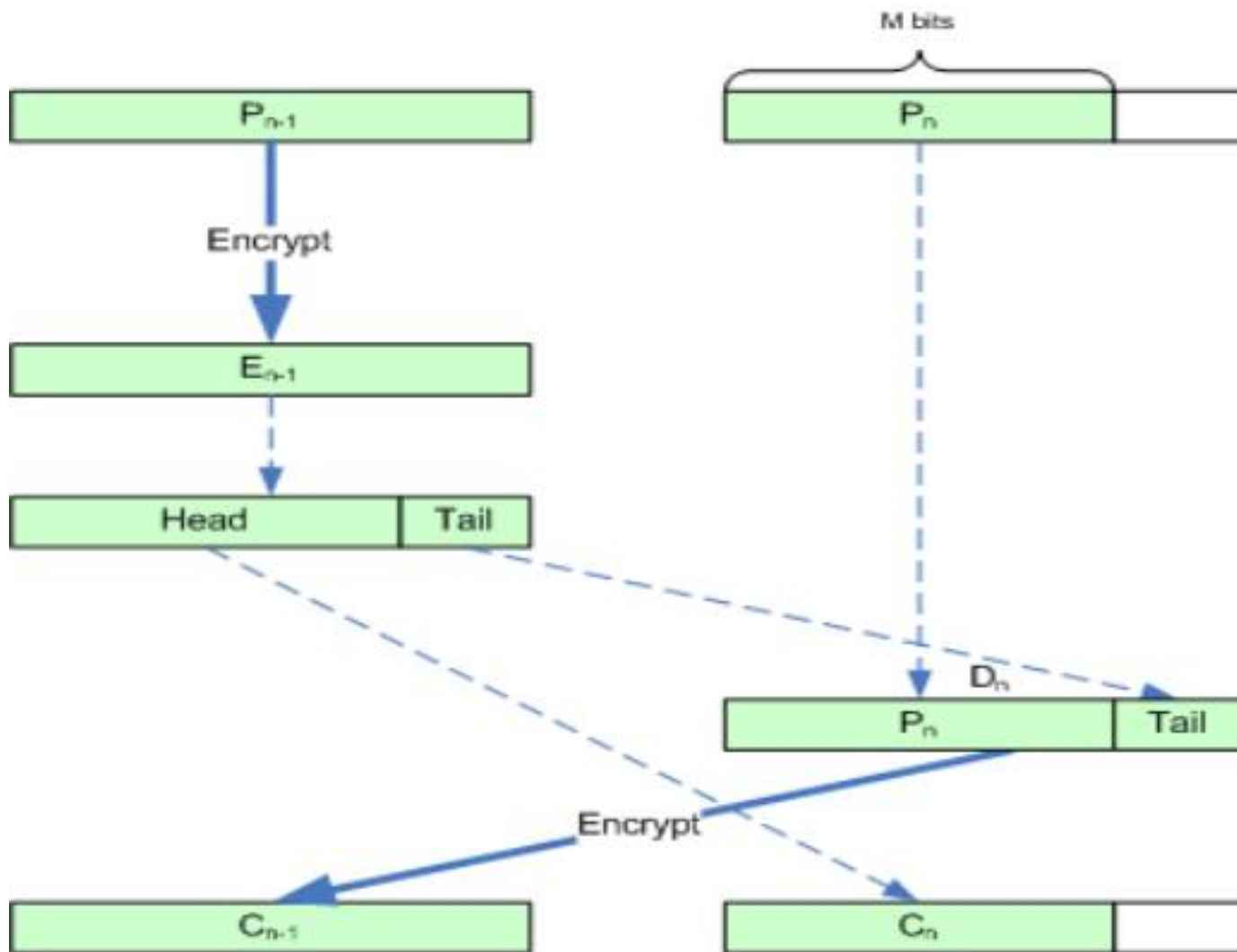
分组密码之填充与密文挪用

- 在使用分组密码时，必须将整个明文分成固定长度的明文分组。当遇到明文长度不是分组长度的倍数时，就必须要对最后一段明文（无法构成一个分组）进行处理。
- 一种思想是对(最后一段)明文进行**Cryptography Padding**(填充)，使明文长度变成分组长度的倍数。填充的思想已经在**Java**和**C#**等高级语言支持的密码类库中得到广泛使用。
- 另一种思想则是进行**Ciphertext stealing**(密文挪用)。密文挪用可以使密文长度与明文长相同，即保证了加密是“原地的”。

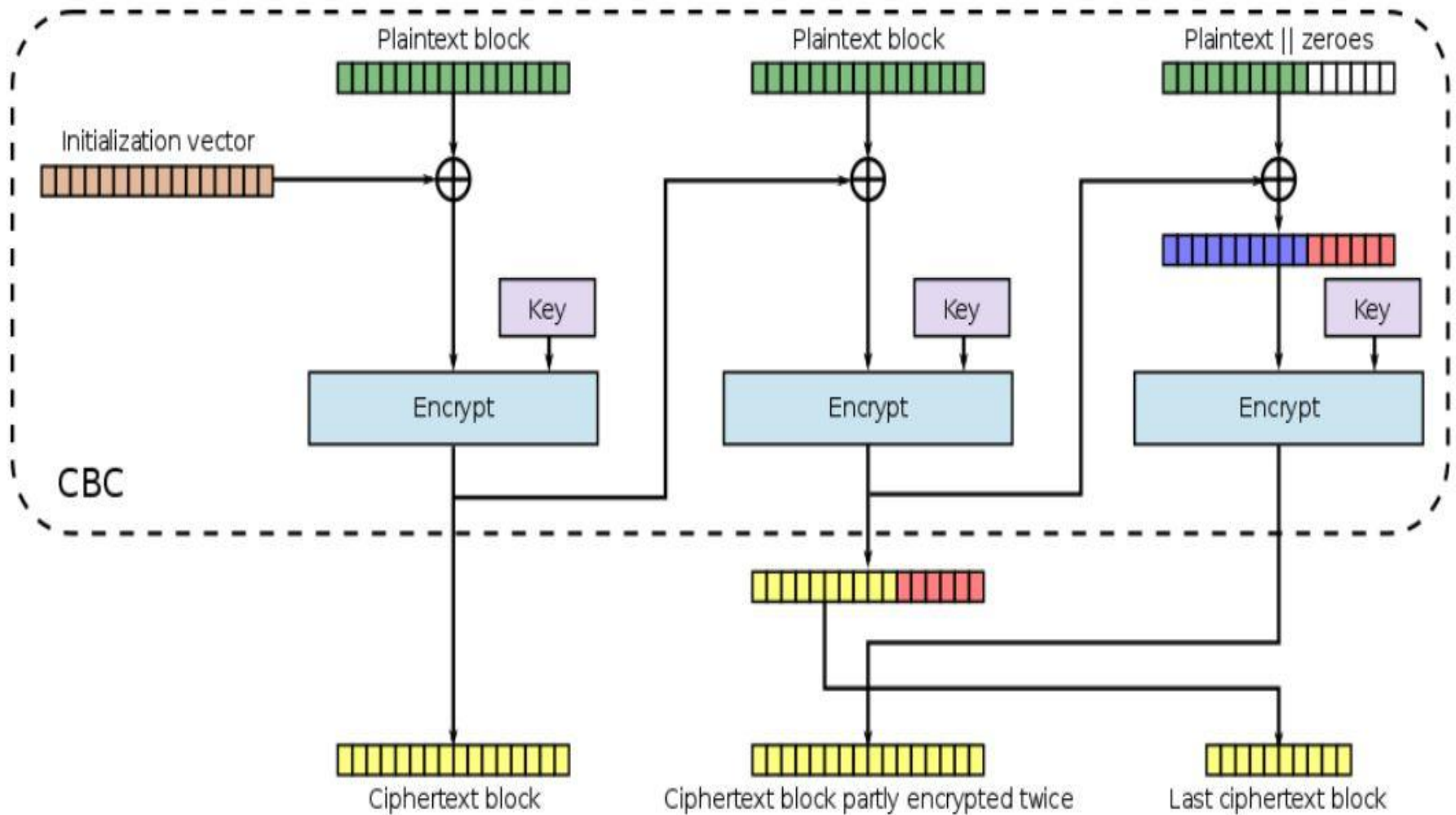
分组密码填充之PKCS#7Padding

- ❑ 填充字符串由一个字节序列组成，每个字节填充该填充字节序列的长度。分组长度为8字节，假设数据长度为9字节。数据为：FF FF FF FF FF FF FF FF FF。
- ❑ PKCS7填充之后为：
FF FF FF FF FF FF FF FF 07 07 07 07 07 07 07
- ① 填充的字节都是一个相同的字节
- ② 填充字节的值就是要填充的字节的个数
- ③ 恰好是8个字节时也要在填充8个字节的08。这可以让解密的数据确定无误的移除多余的字节。
- ❑ 分组密码在CBC工作模式下使用填充技术是不安全的，无法抵御填充预言机攻击（Padding Oracle Attack）

*EBC*模式密文挪用示意图



CBC模式加密密文挪用示意图



CBC模式解密密文挪用示意图

